

ObjektinisProgramavimas-2

2.0

Sugeneruota Doxygen 1.17.0

1 Hierarchijos Indeksas	1
1.1 Klasių hierarchija	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Klasės Dokumentacija	7
4.1 Failai Struktūra	7
4.1.1 Smulkus aprašymas	7
4.2 Studentas Klasė	7
4.2.1 Smulkus aprašymas	9
4.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	9
4.2.2.1 Studentas() [1/6]	9
4.2.2.2 Studentas() [2/6]	10
4.2.2.3 Studentas() [3/6]	11
4.2.2.4 Studentas() [4/6]	11
4.2.2.5 ~Studentas()	11
4.2.2.6 Studentas() [5/6]	12
4.2.2.7 Studentas() [6/6]	12
4.2.3 Metodų Dokumentacija	12
4.2.3.1 addHomeworkGrade()	12
4.2.3.2 calculateFinalGrade()	12
4.2.3.3 getExamGrade()	13
4.2.3.4 getFinalGrade()	13
4.2.3.5 getHomeworkGrades()	13
4.2.3.6 getName()	13
4.2.3.7 getSurname()	14
4.2.3.8 operator=() [1/2]	14
4.2.3.9 operator=() [2/2]	14
4.2.3.10 reserveHomeworkGrades()	14
4.2.3.11 setExamGrade()	15
4.2.3.12 setFinalGrade()	15
4.2.3.13 setHomeworkGrades()	15
4.2.3.14 setName()	15
4.2.3.15 setSurname()	16
4.2.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	16
4.2.4.1 operator<<	16
4.2.4.2 operator>>	16
4.3 StudentasLentelei Struktūra	17
4.3.1 Smulkus aprašymas	17

4.4 TestoLaikai Struktūra	17
4.4.1 Smulkus aprašymas	17
4.5 Zmogus Klasė	18
4.5.1 Smulkus aprašymas	18
4.5.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	18
4.5.2.1 Zmogus() [1/2]	18
4.5.2.2 Zmogus() [2/2]	19
4.5.2.3 ~Zmogus()	20
4.5.3 Metodų Dokumentacija	20
4.5.3.1 getName()	20
4.5.3.2 getSurname()	20
5 Failo Dokumentacija	21
5.1 strukturosFailai/Failai.h Failo Nuoroda	21
5.1.1 Smulkus aprašymas	21
5.2 Failai.h	21
5.3 strukturosFailai/SabloninesFunkcijos.h Failo Nuoroda	22
5.3.1 Smulkus aprašymas	22
5.3.2 Funkcijos Dokumentacija	23
5.3.2.1 apskaiciuotiGalutiniusPazymius()	23
5.3.2.2 nuskaitytiDuomenisGeneric()	23
5.3.2.3 nuskaitytiStudentuDuomenisIsFailo()	23
5.3.2.4 perkeltiStudentus()	24
5.3.2.5 rikiuotiStudentus()	25
5.3.2.6 rikiuotiSuskirstytusStudentus()	25
5.3.2.7 skirstytiIstrinantStudentus()	26
5.3.2.8 skirstytiIstrinantStudentusEfektyviau()	26
5.4 SabloninesFunkcijos.h	27
5.5 strukturosFailai/strukturaDarbasSuFailais.h Failo Nuoroda	28
5.5.1 Smulkus aprašymas	29
5.5.2 Funkcijos Dokumentacija	29
5.5.2.1 gautiTekstiniusFailus()	29
5.5.2.2 irasytiDuomenis()	30
5.5.2.3 irasytiStudentuDuomenisIFaila()	30
5.5.2.4 irasytiSuskirstytusStudentusIFailus()	30
5.5.2.5 nuskaitytiDuomenis()	30
5.5.2.6 nuskaitytiEilutesIVektoriu()	31
5.5.2.7 nuskaitytiStudentuDuomenisIsFailo()	31
5.5.2.8 nuskaitytiSveikaSkaiciusFailo()	31
5.5.2.9 nuskaitytiZodiIsFailo()	32
5.5.2.10 praleistiTarpaisFailo()	32
5.5.2.11 vykdytiSkirstymaIFailus()	32

5.6 strukturaDarbasSuFailais.h	33
5.7 strukturosFailai/strukturaGeneravimas.h Failo Nuoroda	33
5.7.1 Smulkus aprašymas	34
5.7.2 Funkcijos Dokumentacija	34
5.7.2.1 generuotiRezultatus()	34
5.7.2.2 generuotiStudentus()	34
5.7.2.3 generuotiSveikaSkaiciu()	34
5.7.2.4 generuotiVardaPavarde()	35
5.8 strukturaGeneravimas.h	35
5.9 strukturosFailai/strukturalvestis.h Failo Nuoroda	36
5.9.1 Smulkus aprašymas	36
5.9.2 Funkcijos Dokumentacija	36
5.9.2.1 apdorotiIrsvestiStudentus()	36
5.9.2.2 irasytiStudentuDuomenisIFaila()	37
5.9.2.3 isvestiStudentus()	37
5.9.2.4 parodytiRezultatuLentele()	37
5.9.2.5 parodytiStudentus()	38
5.9.2.6 spausdintiVidurkius()	38
5.9.2.7 vykdytiIrasymaIFaila()	38
5.10 strukturalvestis.h	39
5.11 strukturosFailai/strukturalvestis.h Failo Nuoroda	39
5.11.1 Smulkus aprašymas	40
5.11.2 Funkcijos Dokumentacija	40
5.11.2.1 nuskaitytiMenuPasirinkima()	40
5.11.2.2 nuskaitytiNamuDarbuPazymius()	41
5.11.2.3 nuskaitytiNeneigiamaSveikajiSkaiciu()	41
5.11.2.4 nuskaitytiPagrindinioMenuPasirinkima()	41
5.11.2.5 nuskaitytiPazymiNuo1iki10()	42
5.11.2.6 nuskaitytiSkaiciavimoMetoda()	42
5.11.2.7 nuskaitytiSveikajiSkaiciu()	42
5.11.2.8 nuskaitytiTeigiamaSveikajiSkaiciu()	43
5.11.2.9 nuskaitytiVardaArPavarde()	43
5.11.2.10 parinktiRikiavimoBudus()	43
5.11.2.11 patvirtintiNaujoStudentoPridejima()	44
5.11.2.12 saugiaiNuskaitytiEilute()	44
5.11.2.13 suformuotiStudentoIvestiesEilute()	44
5.11.2.14 sukurtiStudentalsEilutesPerOperatoriu()	45
5.11.2.15 tikrintiIvesti()	45
5.11.2.16 tvarkytiPavarde()	45
5.11.2.17 tvarkytiVarda()	45
5.11.2.18 vykdytiPilnaGeneravima()	46
5.11.2.19 vykdytiStudentulvedima()	46

5.12 strukturalvestis.h	46
5.13 struktūrosFailai/strukturaMeniu.h Failo Nuoroda	47
5.13.1 Smulkus aprašymas	48
5.13.2 Funkcijos Dokumentacija	48
5.13.2.1 gautiNuskaitymoMeniu()	48
5.13.2.2 nuskaitytiMeniuPasirinkima() [1/2]	49
5.13.2.3 nuskaitytiMeniuPasirinkima() [2/2]	49
5.13.2.4 nuskaitytiPagrindinioMeniuPasirinkima()	50
5.14 strukturaMeniu.h	50
5.15 struktūrosFailai/strukturaRikiavimas.h Failo Nuoroda	51
5.15.1 Smulkus aprašymas	52
5.15.2 Funkcijos Dokumentacija	52
5.15.2.1 rikiuotiStudentus()	52
5.15.2.2 rikiuotiSuskirstytusStudentus()	52
5.16 strukturaRikiavimas.h	53
5.17 struktūrosFailai/strukturaSkaiciavimai.h Failo Nuoroda	53
5.17.1 Smulkus aprašymas	53
5.17.2 Funkcijos Dokumentacija	54
5.17.2.1 apskaiciuotiGalutiniPazymi()	54
5.17.2.2 apskaiciuotiGalutiniusPazymius()	54
5.17.2.3 lygintiElementusPagalDidejanciaReiksme()	54
5.17.2.4 lygintiElementusPagalMazejanciaReiksme()	55
5.17.2.5 skaiciuotiGalutineMediana()	55
5.17.2.6 skaiciuotiGalutiniVidurki()	55
5.17.2.7 skaiciuotiNDMediana()	56
5.17.2.8 skaiciuotiNDVidurki()	56
5.17.2.9 suskirstytiStudentus()	56
5.18 strukturaSkaiciavimai.h	57
5.19 struktūrosFailai/strukturaTestavimas.h Failo Nuoroda	57
5.19.1 Smulkus aprašymas	59
5.19.2 Funkcijos Dokumentacija	59
5.19.2.1 apskaiciuotiBendraLaika()	59
5.19.2.2 apskaiciuotiLaika()	59
5.19.2.3 gautiVidurki()	59
5.19.2.4 ismatuotiLaika()	60
5.19.2.5 vykdytiIsvedimoTestavima()	60
5.20 strukturaTestavimas.h	60
5.21 struktūrosFailai/strukturaVisi.h Failo Nuoroda	61
5.21.1 Smulkus aprašymas	62
5.22 strukturaVisi.h	62
5.23 Studentas.h Failo Nuoroda	62
5.23.1 Smulkus aprašymas	63

5.23.2 Funkcijos Dokumentacija	63
5.23.2.1 operator<<()	63
5.24 Studentas.h	63
5.25 Zmogus.h Failo Nuoroda	64
5.25.1 Smulkus aprašymas	65
5.26 Zmogus.h	65
Rodyklė	67

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Failai	7
StudentasLentelei	17
TestoLaikai	17
Zmogus	18
Studentas	7

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Failai	Saugo vardų ir pavardžių sąrašus, naudojamus studentų generavimui	7
Studentas	Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį	7
StudentasLentelei	Pagalbinė struktūra studento išvedimui lentelės formatu	17
TestoLaikai	Saugo atskirų programos etapų vykdymo laikus	17
Zmogus	Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti	18

skyrius 3

Failo Indeksas

3.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

Studentas.h	
Studentą aprašanti klasė, paveldinti bazinę Zmogus klasę	62
Zmogus.h	
Abstrakti bazinė žmogaus klasė	64
struktūrosFailai/ Failai.h	
Failų, kuriuose saugomi vardai ir pavardės, nuskaitymo struktūra	21
struktūrosFailai/ SabloninesFunkcijos.h	
Šabloninės funkcijos, leidžiančios apdoroti studentus skirtinguose konteineriuose	22
struktūrosFailai/ strukturaDarbasSuFailais.h	
Funkcijos darbui su tekstiniais failais ir studentų duomenų nuskaitymu bei įrašymu	28
struktūrosFailai/ strukturaGeneravimas.h	
Funkcijos studentų vardams, pavardėms ir pažymiams generuoti	33
struktūrosFailai/ strukturalvestis.h	
Funkcijos studentų duomenų ir testavimo rezultatų išvedimui	36
struktūrosFailai/ strukturalvestis.h	
Funkcijos naudotojo įvesties nuskaitymui, tikrinimui ir apdorojimui	39
struktūrosFailai/ strukturaMeniu.h	
Programos meniu konstantos ir meniu pasirinkimų nuskaitymo funkcijos	47
struktūrosFailai/ strukturaRikiavimas.h	
Funkcijos studentų sąrašams rikiuoti	51
struktūrosFailai/ strukturaSkaiciavimai.h	
Funkcijos studentų pažymių skaičiavimui ir skirstymui	53
struktūrosFailai/ strukturaTestavimas.h	
Programos veikimo greičio, konteinerių strategijų ir Studentas klasės testavimo funkcijos . . .	57
struktūrosFailai/ strukturaVisi.h	
Bendras antraštinis failas, įtraukiantis visus pagrindinius projekto modulius	61

skyrius 4

Klasės Dokumentacija

4.1 Failai Struktūra

Saugo vardų ir pavardžių sąrašus, naudojamus studentų generavimui.

```
#include <Failai.h>
```

Vieši Atributai

- `std::vector< std::string > vyrVardai` = `nuskaitytiEilutesIvektoriu`("VardulrPavardziuSarasai/Lietuviski_vyru_vardai.txt")
- `std::vector< std::string > vyrPavardes` = `nuskaitytiEilutesIvektoriu`("VardulrPavardziuSarasai/Lietuviskos_vyru_pavardes.txt")
- `std::vector< std::string > motVardai` = `nuskaitytiEilutesIvektoriu`("VardulrPavardziuSarasai/Lietuviski_moteru_vardai.txt")
- `std::vector< std::string > motPavardes` = `nuskaitytiEilutesIvektoriu`("VardulrPavardziuSarasai/Lietuviskos_moteru_pavardes.txt")

4.1.1 Smulkus aprašymas

Saugo vardų ir pavardžių sąrašus, naudojamus studentų generavimui.

Struktūra automatiškai inicijuoja keturis vektorius, nuskaitydama duomenis iš nurodytų tekstinių failų.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

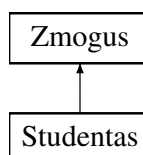
- `strukturosFailai/Failai.h`

4.2 Studentas Klasė

Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.

```
#include <Studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- **Studentas** ()
Numatytasis konstruktorius.
- **Studentas** (const std::string &name, const std::string &surname, int examGrade, const std::vector< int > &homeworkGrades, double finalGrade)
Konstruktorius su visais pagrindiniais studento duomenimis.
- **Studentas** (const char *eilute, std::size_t namuDarbuKiekis)
Sukuria studentą iš tekstinės eilutės.
- **Studentas** (bool generuotiPazymius, int ndKiekis)
Sukuria studentą pagal generavimo nustatymus.
- **~Studentas** ()
Destruktorius.
- **Studentas** (const **Studentas** &other)
Kopijavimo konstruktorius.
- **Studentas** & **operator=** (const **Studentas** &other)
Kopijavimo priskyrimo operatorius.
- **Studentas** (**Studentas** &&other) noexcept
Perkėlimo konstruktorius.
- **Studentas** & **operator=** (**Studentas** &&other) noexcept
Perkėlimo priskyrimo operatorius.
- std::string **getName** () const
Grąžina studento vardą.
- std::string **getSurname** () const
Grąžina studento pavardę.
- double **getFinalGrade** () const
Grąžina studento galutinį pažymį.
- int **getExamGrade** () const
Grąžina studento egzamino pažymį.
- const std::vector< int > & **getHomeworkGrades** () const
Grąžina studento namų darbų pažymių sąrašą.
- double **calculateFinalGrade** (char method) const
Apskaičiuoja studento galutinį pažymį pagal pasirinktą metodą.
- void **setName** (const std::string &name)
Nustato studento vardą.
- void **setSurname** (const std::string &surname)
Nustato studento pavardę.
- void **setExamGrade** (int examGrade)
Nustato egzamino pažymį.
- void **setFinalGrade** (double finalGrade)
Nustato galutinį pažymį.
- void **setHomeworkGrades** (const std::vector< int > &homeworkGrades)
Nustato visą namų darbų pažymių sąrašą.
- void **addHomeworkGrade** (int grade)
Prideda vieną namų darbo pažymį.
- void **clearHomeworkGrades** ()
Išvalo visus namų darbų pažymius.
- void **reserveHomeworkGrades** (std::size_t n)
Rezervuoja vietą namų darbų pažymių vektoriuje.

Vieši Metodai inherited from Zmogus

- `Zmogus()`=default
Numatytasis konstruktorius.
- `Zmogus(const std::string &name, const std::string &surname)`
Konstruktorius su vardu ir pavarde.
- `virtual ~Zmogus()`=0
Grynas virtualus destruktorius.

Statiniai Vieši Atributai

- static bool `arSpausdintiDestruktoriu`
Nurodo, ar destruktorius turi spausdinti informaciją pranešimą.

Draugai

- `std::ostream & operator<< (std::ostream &os, const Studentas &s)`
Išveda studento duomenis į srautą.
- `std::istream & operator>> (std::istream &is, Studentas &s)`
Nuskaityto studento duomenis iš įvesties srauto.

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `std::string name_`
Žmogaus vardas.
- `std::string surname_`
Žmogaus pavardė.

4.2.1 Smulkus aprašymas

Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.

Klasė paveldi abstrakčią bazinę klasę `Zmogus`. Ji realizuoja vardo ir pavardės gavimo metodus, saugo pažymių informaciją ir pateikia metodus studento duomenims keisti bei galutiniam pažymiui apskaičiuoti.

4.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.2.2.1 Studentas() [1/6]

```
Studentas::Studentas () [inline]
```

Numatytasis konstruktorius.

Sukuria tuščią studentą su nuliniu egzamino ir galutiniu pažymiu.

4.2.2.2 Studentas() [2/6]

```
Studentas::Studentas (
    const std::string & name,
    const std::string & surname,
    int examGrade,
    const std::vector< int > & homeworkGrades,
    double finalGrade) [inline]
```

Konstruktorius su visais pagrindiniais studento duomenimis.

Parametrai

<i>name</i>	Studento vardas.
-------------	------------------

<i>surname</i>	Studento pavardė.
<i>examGrade</i>	Egzamino pažymys.
<i>homeworkGrades</i>	Namų darbų pažymių vektorius.
<i>finalGrade</i>	Galutinis pažymys.

4.2.2.3 Studentas() [3/6]

```
Studentas::Studentas (
    const char * eilute,
    std::size_t namuDarbuKiekis)
```

Sukuria studentą iš tekstinės eilutės.

Konstruktorius naudojamas nuskaitant studentų duomenis iš failo.

Parametrai

<i>eilute</i>	Simbolių eilutė su studento duomenimis.
<i>namuDarbuKiekis</i>	Namų darbų pažymių kiekis eilutėje.

4.2.2.4 Studentas() [4/6]

```
Studentas::Studentas (
    bool generuotiPazymius,
    int ndKiekis)
```

Sukuria studentą pagal generavimo nustatymus.

Parametrai

<i>generuotiPazymius</i>	Nurodo, ar pažymiai turi būti sugeneruoti automatiškai.
<i>ndKiekis</i>	Namų darbų pažymių kiekis.

4.2.2.5 ~Studentas()

```
Studentas::~~Studentas ()
```

Destruktorius.

Jei [arSpausdintiDestruktoriu](#) reikšmė yra `true`, gali būti spausdinamas informacinis pranešimas apie objekto sunaikinimą.

4.2.2.6 Studentas() [5/6]

```
Studentas::Studentas (  
    const Studentas & other)
```

Kopijavimo konstruktorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio kopijuojami duomenys.
--------------	---

4.2.2.7 Studentas() [6/6]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Perkėlimo konstruktorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio perkeliama duomenys.
--------------	--

4.2.3 Metodų Dokumentacija

4.2.3.1 addHomeworkGrade()

```
void Studentas::addHomeworkGrade (  
    int grade) [inline]
```

Prideda vieną namų darbo pažymį.

Parametrai

<i>grade</i>	Pridedamas pažymys.
--------------	---------------------

4.2.3.2 calculateFinalGrade()

```
double Studentas::calculateFinalGrade (  
    char method) const
```

Apskaičiuoja studento galutinį pažymį pagal pasirinktą metodą.

Metodas paprastai nurodo, ar galutinis pažymys skaičiuojamas pagal namų darbų vidurkį, ar pagal medianą.

Parametrai

<i>method</i>	Skaiciavimo metodo simbolis.
---------------	------------------------------

Gražina

Apskaičiuotas galutinis pažymys.

4.2.3.3 getExamGrade()

```
int Studentas::getExamGrade () const [inline]
```

Grąžina studento egzamino pažymį.

Gražina

Egzamino pažymys.

4.2.3.4 getFinalGrade()

```
double Studentas::getFinalGrade () const [inline]
```

Grąžina studento galutinį pažymį.

Gražina

Galutinis pažymys.

4.2.3.5 getHomeworkGrades()

```
const std::vector< int > & Studentas::getHomeworkGrades () const [inline]
```

Grąžina studento namų darbų pažymių sąrašą.

Gražina

Konstantinė nuoroda į namų darbų pažymių vektorių.

4.2.3.6 getName()

```
std::string Studentas::getName () const [inline], [virtual]
```

Grąžina studento vardą.

Gražina

Studento vardas.

Realizuoja [Zmogus](#).

4.2.3.7 getSurname()

```
std::string Studentas::getSurname () const [inline], [virtual]
```

Gražina studento pavardę.

Gražina

Studento pavardė.

Realizuoja [Zmogus](#).

4.2.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio kopijuojami duomenys.
--------------	---

Gražina

Nuoroda į esamą objektą.

4.2.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio perkeliama duomenys.
--------------	--

Gražina

Nuoroda į esamą objektą.

4.2.3.10 reserveHomeworkGrades()

```
void Studentas::reserveHomeworkGrades (
    std::size_t n) [inline]
```

Rezervuoja vietą namų darbų pažymių vektoriuje.

Parametrai

<i>n</i>	Rezervuojamas elementų kiekis.
----------	--------------------------------

4.2.3.11 setExamGrade()

```
void Studentas::setExamGrade (
    int examGrade) [inline]
```

Nustato egzamino pažymį.

Parametrai

<i>examGrade</i>	Naujas egzamino pažymys.
------------------	--------------------------

4.2.3.12 setFinalGrade()

```
void Studentas::setFinalGrade (
    double finalGrade) [inline]
```

Nustato galutinį pažymį.

Parametrai

<i>finalGrade</i>	Naujas galutinis pažymys.
-------------------	---------------------------

4.2.3.13 setHomeworkGrades()

```
void Studentas::setHomeworkGrades (
    const std::vector< int > & homeworkGrades) [inline]
```

Nustato visą namų darbų pažymių sąrašą.

Parametrai

<i>homeworkGrades</i>	Naujas namų darbų pažymių vektorius.
-----------------------	--------------------------------------

4.2.3.14 setName()

```
void Studentas::setName (
    const std::string & name) [inline]
```

Nustato studento vardą.

Parametrai

<i>name</i>	Naujas vardas.
-------------	----------------

4.2.3.15 setSurname()

```
void Studentas::setSurname (
    const std::string & surname) [inline]
```

Nustato studento pavardę.

Parametrai

<i>surname</i>	Nauja pavardė.
----------------	----------------

4.2.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.2.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Išveda studento duomenis į srautą.

Parametrai

<i>os</i>	Išvesties srautas.
<i>s</i>	Studentas , kurio duomenys išvedami.

Gražina

Išvesties srautas.

4.2.4.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Nuskaito studento duomenis iš įvesties srauto.

Parametrai

<i>is</i>	Įvesties srautas.
<i>s</i>	Studentas , į kurį įrašomi duomenys.

Gražina

Įvesties srautas.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [Studentas.h](#)

4.3 StudentasLentelei Struktūra

Pagalbinė struktūra studento išvedimui lentelės formatu.

```
#include <Studentas.h>
```

Vieši Atributai

- const [Studentas](#) & **s**
[Studentas](#), kuris bus išvedamas lentelės formatu.

4.3.1 Smulkus aprašymas

Pagalbinė struktūra studento išvedimui lentelės formatu.

Struktūra saugo konstantinę nuorodą į studentą ir naudojama specializuotam išvedimo operatoriui.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [Studentas.h](#)

4.4 TestoLaikai Struktūra

Saugo atskirų programos etapų vykdymo laikus.

```
#include <strukturaTestavimas.h>
```

Vieši Atributai

- double **nuskaitymas** = 0.0
Duomenų nuskaitymo laikas sekundėmis.
- double **skaiciavimas** = 0.0
Galutinių pažymių skaičiavimo laikas sekundėmis.
- double **rikiavimas** = 0.0
Studentų rikiavimo laikas sekundėmis.
- double **skirstymas** = 0.0
Studentų skirstymo laikas sekundėmis.
- double **isvedimas** = 0.0
Rezultatų išvedimo laikas sekundėmis.

4.4.1 Smulkus aprašymas

Saugo atskirų programos etapų vykdymo laikus.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

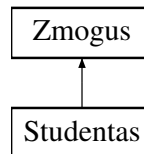
- strukturosFailai/[strukturaTestavimas.h](#)

4.5 Zmogus Klasė

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus()`=default
Numatytasis konstruktorius.
- `Zmogus(const std::string &name, const std::string &surname)`
Konstruktorius su vardu ir pavarde.
- `virtual ~Zmogus()`=0
Grynas virtualus destruktorius.
- `virtual std::string getName()` const =0
Grąžina žmogaus vardą.
- `virtual std::string getSurname()` const =0
Grąžina žmogaus pavardę.

Apsaugoti Atributai

- `std::string name_`
Žmogaus vardas.
- `std::string surname_`
Žmogaus pavardė.

4.5.1 Smulkus aprašymas

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

Klasė turi grynai virtualius metodus vardui ir pavardei gauti. Iš jos paveldi konkretesnės klasės, pavyzdžiui, `Studentas`.

4.5.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.5.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [default]
```

Numatytasis konstruktorius.

Sukuria tuščią bazinės klasės objektą.

4.5.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & name,
    const std::string & surname) [inline]
```

Konstruktorius su vardu ir pavarde.

Parametrai

<i>name</i>	Žmogaus vardas.
-------------	-----------------

<i>surname</i>	Žmogaus pavardė.
----------------	------------------

4.5.2.3 ~Zmogus()

```
Zmogus::~~Zmogus () [inline], [pure virtual]
```

Grynas virtualus destruktorius.

Gryno virtualaus destruktoriaus apibrėžimas.

Destruktorius yra virtualus, kad išvestinių klasių objektai būtų korektiškai sunaikinami per bazinės klasės rodyklę.

Nors destruktorius yra grynai virtualus, jis vis tiek turi turėti apibrėžimą, kuris pateiktas žemiau kaip `inline`.

Grynas virtualus destruktorius privalo turėti apibrėžimą, nes jis vis tiek iškviečiamas naikinant išvestinės klasės objektą.

4.5.3 Metodų Dokumentacija

4.5.3.1 getName()

```
virtual std::string Zmogus::getName () const [pure virtual]
```

Grąžina žmogaus vardą.

Gražina

Žmogaus vardas.

Realizuota [Studentas](#).

4.5.3.2 getSurname()

```
virtual std::string Zmogus::getSurname () const [pure virtual]
```

Grąžina žmogaus pavardę.

Gražina

Žmogaus pavardė.

Realizuota [Studentas](#).

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [Zmogus.h](#)

skyrius 5

Failo Dokumentacija

5.1 struktūrosFailai/Failai.h Failo Nuoroda

Failų, kuriuose saugomi vardai ir pavardės, nuskaitymo struktūra.

```
#include <vector>
#include <string>
#include "../strukturosFailai/strukturaDarbasSuFailais.h"
```

Klasės

- struct [Failai](#)

Saugo vardų ir pavardžių sąrašus, naudojamus studentų generavimui.

5.1.1 Smulkus aprašymas

Failų, kuriuose saugomi vardai ir pavardės, nuskaitymo struktūra.

Šiame faile aprašoma struktūra [Failai](#), kuri saugo lietuviškų vyrų ir moterų vardų bei pavardžių sąrašus. Duomenys nuskaityti iš tekstinių failų naudojant funkciją [nuskaitytiEilutesIVektoriu](#).

5.2 Failai.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef FAILAI_H
00002 #define FAILAI_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include "../strukturosFailai/strukturaDarbasSuFailais.h"
00007
00016
00024 struct Failai{
00025     std::vector<std::string> vyrVardai =
00026         nuskaitytiEilutesIVektoriu("VarduIrPavardziuSarasai/Lietuviski_vyru_vardai.txt");
00026     std::vector<std::string> vyrPavardes =
00027         nuskaitytiEilutesIVektoriu("VarduIrPavardziuSarasai/Lietuviskos_vyru_pavardes.txt");
00027     std::vector<std::string> motVardai =
00028         nuskaitytiEilutesIVektoriu("VarduIrPavardziuSarasai/Lietuviski_moteru_vardai.txt");
00028     std::vector<std::string> motPavardes =
00029         nuskaitytiEilutesIVektoriu("VarduIrPavardziuSarasai/Lietuviskos_moteru_pavardes.txt");
00029 };
00030
00031 #endif
```

5.3 struktūrosFailai/SabloninesFunkcijos.h Failo Nuoroda

Šabloninės funkcijos, leidžiančios apdoroti studentus skirtinguose konteineriuose.

```
#include <string>
#include <algorithm>
#include <numeric>
#include <iostream>
#include "../struktūrosFailai/strukturaDarbasSuFailais.h"
#include <fstream>
#include <list>
#include <type_traits>
#include <utility>
```

Funkcijos

- `template<typename StudentuKonteineris>`
`StudentuKonteineris nuskaitytiStudentuDuomenisIsFailo` (const std::string &failas)
Nuskaityti studentų duomenis iš failo į pasirinktą konteinerį.
- `template<typename StudentuKonteineris>`
`void rikiuotiStudentus` (int pasirinkimasRikiavimo, StudentuKonteineris &studentuSarasas)
Rikiuoja studentus pagal pasirinktą kriterijų.
- `template<typename StudentuKonteineris>`
`void apskaiciuotiGalutiniusPazymius` (StudentuKonteineris &duomenys, char metodas)
Apskaičiuoja galutinį pažymį visiems studentams konteineryje.
- `template<typename SaltinioKonteineris, typename RezultatoKonteineris>`
`void perkeltiStudentus` (SaltinioKonteineris &studentai, RezultatoKonteineris &pazangusStudentai, RezultatoKonteineris &silpniStudentai)
Perkelia studentus į pažangių ir silpnų studentų konteinerius.
- `template<typename SaltinioKonteineris, typename RezultatoKonteineris>`
`void skirstytiIstirinantStudentus` (SaltinioKonteineris &studentai, RezultatoKonteineris &silpniStudentai)
Išskiria silpnus studentus iš pradinio konteinerio naudojant `std::partition`.
- `template<typename SaltinioKonteineris, typename RezultatoKonteineris>`
`void skirstytiIstirinantStudentusEfektyviau` (SaltinioKonteineris &studentai, RezultatoKonteineris &silpniStudentai)
Išskiria silpnus studentus naudojant `std::remove_if`.
- `template<typename StudentuKonteineris>`
`void nuskaitytiDuomenisGeneric` (int pasirinkimasNuskaitymo, StudentuKonteineris &studentuSarasas, std::string &katalogas)
Nuskaityti studentų duomenis iš pasirinkto katalogo failo į pasirinktą konteinerį.
- `template<typename StudentuKonteineris>`
`void rikiuotiSuskirstytusStudentus` (StudentuKonteineris &pazangiuSarasas, StudentuKonteineris &silpniSarasas, int pasirinkimasRikiavimoPazangiu, int pasirinkimasRikiavimoSilpni)
Rikiuoja pažangių ir silpnų studentų konteinerius pagal pasirinktus kriterijus.

5.3.1 Smulkus aprašymas

Šabloninės funkcijos, leidžiančios apdoroti studentus skirtinguose konteineriuose.

Funkcijos pritaikytos darbui su tokiais konteineriais kaip `std::vector` ir `std::list`. Kai įmanoma, naudojamos konteinerio specifinės operacijos, pavyzdžiui, `list::sort`.

5.3.2 Funkcijos Dokumentacija

5.3.2.1 apskaiciuotiGalutiniusPazymius()

```
template<typename StudentuKonteineris>
void apskaiciuotiGalutiniusPazymius (
    StudentuKonteineris & duomenys,
    char metodas)
```

Apskaičiuoja galutinį pažymį visiems studentams konteineryje.

Template Parameters

<i>StudentuKonteineris</i>	Studentų konteinerio tipas.
----------------------------	-----------------------------

Parametrai

<i>duomenys</i>	Studentų konteineris.
<i>metodas</i>	Skaičiavimo metodas, pvz. vidurkis arba mediana.

5.3.2.2 nuskaitytiDuomenisGeneric()

```
template<typename StudentuKonteineris>
void nuskaitytiDuomenisGeneric (
    int pasirinkimasNuskaitymo,
    StudentuKonteineris & studentuSarasas,
    std::string & katalogas)
```

Nuskaito studentų duomenis iš pasirinkto katalogo failo į pasirinktą konteinerį.

Template Parameters

<i>StudentuKonteineris</i>	Studentų konteinerio tipas.
----------------------------	-----------------------------

Parametrai

<i>pasirinkimasNuskaitymo</i>	Pasirinkto failo numeris.
<i>studentuSarasas</i>	Studentų konteineris, į kurį bus įrašyti duomenys.
<i>katalogas</i>	Katalogas, kuriame ieškoma .txt failų.

5.3.2.3 nuskaitytiStudentuDuomenisIsFailo()

```
template<typename StudentuKonteineris>
StudentuKonteineris nuskaitytiStudentuDuomenisIsFailo (
    const std::string & failas)
```

Nuskaityto studentų duomenis iš failo į pasirinktą konteinerį.

Funkcija nuskaityto failo antraštę, pagal ją nustato namų darbų kiekį, o kiekvieną kitą eilutę bando konvertuoti į `Studentas` objektą. Nekorektiškos eilutės praleidžiamos.

Template Parameters

<i>StudentuKonteineris</i>	Konteinerio tipas, pvz. <code>std::vector<Studentas></code> arba <code>std::list<Studentas></code> .
----------------------------	--

Parametrai

<i>failas</i>	Failo kelias.
---------------	---------------

Gražina

Konteineris su nuskaitytais studentais.

Išimtys

<i>std::runtime_error</i>	Jei failo nepavyksta atidaryti.
---------------------------	---------------------------------

5.3.2.4 perkeltiStudentus()

```
template<typename SaltinioKonteineris, typename RezultatoKonteineris>
void perkeltiStudentus (
    SaltinioKonteineris & studentai,
    RezultatoKonteineris & pazangusStudentai,
    RezultatoKonteineris & silpniStudentai)
```

Perkelia studentus į pažangių ir silpnų studentų konteinerius.

Studentai, kurių galutinis pažymys mažesnis nei 5.0, perkeliama į silpnų studentų sąrašą. Kiti studentai perkeliama į pažangių studentų sąrašą.

Template Parameters

<i>SaltinioKonteineris</i>	Pradinio studentų konteinerio tipas.
<i>RezultatoKonteineris</i>	Rezultato konteinerio tipas.

Parametrai

<i>studentai</i>	Pradinis studentų konteineris.
<i>pazangusStudentai</i>	Konteineris pažangiems studentams.
<i>silpniStudentai</i>	Konteineris silpniems studentams.

5.3.2.5 rikiuotiStudentus()

```
template<typename StudentuKonteineris>
void rikiuotiStudentus (
    int pasirinkimasRikiavimo,
    StudentuKonteineris & studentuSarasas)
```

Rikiuoja studentus pagal pasirinktą kriterijų.

Pasirinkimai:

- 1 — pagal vardą;
- 2 — pagal pavardę;
- 3 — pagal galutinį pažymį.

Jei konteineris turi metodą `sort`, naudojamas jis. Kitu atveju naudojama `std::sort`.

Template Parameters

<i>StudentuKonteineris</i>	Studentų konteinerio tipas.
----------------------------	-----------------------------

Parametrai

<i>pasirinkimasRikiavimo</i>	Rikiavimo kriterijaus numeris.
<i>studentuSarasas</i>	Studentų konteineris, kuris bus rikiuojamas.

5.3.2.6 rikiuotiSuskirstytusStudentus()

```
template<typename StudentuKonteineris>
void rikiuotiSuskirstytusStudentus (
    StudentuKonteineris & pazangiuSarasas,
    StudentuKonteineris & silpnuSarasas,
    int pasirinkimasRikiavimoPazangiu,
    int pasirinkimasRikiavimoSilpnu)
```

Rikiuoja pažangių ir silpnų studentų konteinerius pagal pasirinktus kriterijus.

Template Parameters

<i>StudentuKonteineris</i>	Studentų konteinerio tipas.
----------------------------	-----------------------------

Parametrai

<i>pazangiuSarasas</i>	Pažangių studentų konteineris.
<i>silpnuSarasas</i>	Silpnų studentų konteineris.
<i>pasirinkimasRikiavimoPazangiu</i>	Pažangių studentų rikiavimo kriterijus.

<i>pasirinkimasRikiavimoSilpnu</i>

Silpnų studentų rikiavimo kriterijus.

5.3.2.7 skirstytiIstrinantStudentus()

```
template<typename SaltinioKonteineris, typename RezultatoKonteineris>
void skirstytiIstrinantStudentus (
    SaltinioKonteineris & studentai,
    RezultatoKonteineris & silpniStudentai)
```

Išskiria silpnus studentus iš pradinio konteinerio naudojant `std::partition`.

Po funkcijos vykdymo pradiniame konteinyje lieka tik pažangūs studentai, o silpni studentai perkeliama į atskirą konteinerį.

Template Parameters

<i>SaltinioKonteineris</i>	Pradinio studentų konteinerio tipas.
<i>RezultatoKonteineris</i>	Rezultato konteinerio tipas.

Parametrai

<i>studentai</i>	Pradinis studentų konteineris.
<i>silpniStudentai</i>	Konteineris silpniems studentams.

5.3.2.8 skirstytiIstrinantStudentusEfektyviau()

```
template<typename SaltinioKonteineris, typename RezultatoKonteineris>
void skirstytiIstrinantStudentusEfektyviau (
    SaltinioKonteineris & studentai,
    RezultatoKonteineris & silpniStudentai)
```

Išskiria silpnus studentus naudojant `std::remove_if`.

Funkcija nukopijuoja silpnus studentus į rezultatų konteinerį ir pašalina juos iš pradinio konteinerio.

Template Parameters

<i>SaltinioKonteineris</i>	Pradinio studentų konteinerio tipas.
<i>RezultatoKonteineris</i>	Rezultato konteinerio tipas.

Parametrai

<i>studentai</i>	Pradinis studentų konteineris.
<i>silpniStudentai</i>	Konteineris silpniems studentams.

5.4 SabloninesFunkcijos.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef SABLONINESFUNKCIJOS_H
00002 #define SABLONINESFUNKCIJOS_H
00003
00004 #include <string>
00005 #include <algorithm>
00006 #include <numeric>
00007 #include <iostream>
00008 #include "../strukturosFailai/strukturaDarbasSuFailais.h"
00009 #include <fstream>
00010 #include <list>
00011 #include <type_traits>
00012 #include <utility>
00013
00021
00035 template <typename StudentuKonteineris>
00036 StudentuKonteineris nuskaitytiStudentuDuomenisIsFailo(const std::string& failas) {
00037     StudentuKonteineris studentuSarasas;
00038     FILE* skaitomasFailas = std::fopen(failas.c_str(), "r");
00039     if (!skaitomasFailas) throw std::runtime_error("Nepavyko atidaryti failo: " + failas);
00040     try {
00041         static char ivestiesBuferis[1 « 20];
00042         std::setvbuf(skaitomasFailas, ivestiesBuferis, _IOFBF, sizeof(ivistiesBuferis));
00043         char antraste[257];
00044         if (!std::fgets(antraste, sizeof(antraste), skaitomasFailas)) {
00045             std::fclose(skaitomasFailas);
00046             return studentuSarasas;
00047         }
00048         std::size_t namuDarbuKiekis = 0;
00049         char laikinaEilute[257];
00050         std::strncpy(laikinaEilute, antraste, sizeof(laikinaEilute));
00051         laikinaEilute[sizeof(laikinaEilute) - 1] = '\0';
00052         char* stulpelis = std::strtok(laikinaEilute, " \t\r\n");
00053         while (stulpelis) {
00054             if (stulpelis[0] == 'N' && stulpelis[1] == 'D') namuDarbuKiekis++;
00055             stulpelis = std::strtok(nullptr, " \t\r\n");
00056         }
00057         char eilute[1024];
00058         while (std::fgets(eilute, sizeof(eilute), skaitomasFailas)) {
00059             try {
00060                 Studentas studentas(eilute, namuDarbuKiekis);
00061                 studentuSarasas.push_back(std::move(studentas));
00062             } catch (...) {
00063                 continue;
00064             }
00065         }
00066         std::fclose(skaitomasFailas);
00067         return studentuSarasas;
00068     }
00069     catch (const std::bad_alloc&) {
00070         std::fclose(skaitomasFailas);
00071         std::cerr « "Nepakanka atminties...\n";
00072         return studentuSarasas;
00073     }
00074 }
00075
00090 template <typename StudentuKonteineris>
00091 void rikiuotiStudentus(int pasirinkimasRikiavimo, StudentuKonteineris& studentuSarasas) {
00092     using StudentasTipas = typename StudentuKonteineris::value_type;
00093     auto rikiuoti = [&](auto comparator) {
00094         if constexpr (requires { studentuSarasas.sort(comparator); })
00095             studentuSarasas.sort(comparator);
00096         else std::sort(studentuSarasas.begin(), studentuSarasas.end(), comparator);
00097     };
00098     if (pasirinkimasRikiavimo == 1)
00099         rikiuoti(lygintiElementusPagalDidejanciaReiksme(&StudentasTipas::getName));
00100     else if (pasirinkimasRikiavimo == 2)
00101         rikiuoti(lygintiElementusPagalDidejanciaReiksme(&StudentasTipas::getSurname));
00102     else if (pasirinkimasRikiavimo == 3)
00103         rikiuoti(lygintiElementusPagalDidejanciaReiksme(&StudentasTipas::getFinalGrade));
00104 }
00105
00108 template <typename StudentuKonteineris>
00109 void apskaiciuotiGalutiniusPazymius(StudentuKonteineris& duomenys, char metodas) {
00110     for (auto& studentas : duomenys) studentas.setFinalGrade(studentas.calculateFinalGrade(metodas));
00111 }
00112
00125 template <typename SaltinioKonteineris, typename RezultatoKonteineris>
00126 void perkeltiStudentus(SaltinioKonteineris& studentai,
00127     RezultatoKonteineris& pazangusStudentai,
00128     RezultatoKonteineris& silpniStudentai) {
00129     pazangusStudentai.clear();
00130     silpniStudentai.clear();

```

```

00131     if constexpr (requires { pazangusStudentai.reserve(studentai.size() / 2); })
pazangusStudentai.reserve(studentai.size() / 2);
00132     if constexpr (requires { silpniStudentai.reserve(studentai.size() / 2); })
silpniStudentai.reserve(studentai.size() / 2);
00133     for (auto& studentas : studentai) {
00134         if (studentas.getFinalGrade() < 5.0) silpniStudentai.push_back(std::move(studentas));
00135         else pazangusStudentai.push_back(std::move(studentas));
00136     }
00137 }
00138
00150 template <typename SaltinioKonteineris, typename RezultatoKonteineris>
00151 void skirstytiIstrinantStudentus(SaltinioKonteineris& studentai,
00152                                 RezultatoKonteineris& silpniStudentai) {
00153     silpniStudentai.clear();
00154     if constexpr (requires { silpniStudentai.reserve(studentai.size() / 2); })
silpniStudentai.reserve(studentai.size() / 2);
00155     auto it = std::partition(studentai.begin(), studentai.end(), [](const Studentas& studentas) {
return studentas.getFinalGrade() < 5.0; });
00156     silpniStudentai.insert(silpniStudentai.end(), std::make_move_iterator(studentai.begin()),
std::make_move_iterator(it));
00157     studentai.erase(studentai.begin(), it);
00158 }
00159
00171 template <typename SaltinioKonteineris, typename RezultatoKonteineris>
00172 void skirstytiIstrinantStudentusEfektyviau(SaltinioKonteineris& studentai, RezultatoKonteineris&
silpniStudentai) {
00173     silpniStudentai.clear();
00174     if constexpr (requires { silpniStudentai.reserve(studentai.size() / 2); })
silpniStudentai.reserve(studentai.size() / 2);
00175     auto it = std::remove_if(studentai.begin(), studentai.end(), [&](const Studentas& studentas) {
00176         if (studentas.getFinalGrade() < 5.0) {
00177             silpniStudentai.push_back(studentas);
00178             return true;
00179         }
00180         return false;
00181     });
00182     studentai.erase(it, studentai.end());
00183 }
00184
00192 template <typename StudentuKonteineris>
00193 void nuskaitytiDuomenisGeneric(int pasirinkimasNuskaitymo, StudentuKonteineris& studentuSarasas,
std::string& katalogas) {
00194     try {
00195         auto failai = gautiTekstiniusFailus(katalogas);
00196         if (failai.empty()) throw std::runtime_error("Kataloge " + katalogas + " nerasta .txt
failų.");
00197         if (pasirinkimasNuskaitymo < 1 ||
00198             static_cast<std::size_t>(pasirinkimasNuskaitymo) > failai.size()) {
00199             throw std::runtime_error("Neteisingas failo pasirinkimas.");
00200         }
00201         const std::size_t indeksas = static_cast<std::size_t>(pasirinkimasNuskaitymo - 1);
00202         const std::string failoKelias = failai[indeksas].string();
00203         studentuSarasas = nuskaitytiStudentuDuomenisIsFailo<StudentuKonteineris>(failoKelias);
00204     }
00205     catch (const std::exception& e) {
00206         std::cerr << "Klaida skaitant failą: " << e.what() << std::endl;
00207         studentuSarasas.clear();
00208     }
00209 }
00210
00219 template <typename StudentuKonteineris>
00220 void rikiuotiSusikirstytusStudentus(StudentuKonteineris& pazangiuSarasas, StudentuKonteineris&
silpnuSarasas, int pasirinkimasRikiavimoPazangiu, int pasirinkimasRikiavimoSilpnu) {
00221     rikiuotiStudentus(pasirinkimasRikiavimoPazangiu, pazangiuSarasas);
00222     rikiuotiStudentus(pasirinkimasRikiavimoSilpnu, silpnuSarasas);
00223 }
00224
00225 #endif

```

5.5 struktūros Failai/struktūra Darbas Su Failais.h Failo Nuoroda

Funkcijos darbui su tekstiniais failais ir studentų duomenų nuskaitymu bei įrašymu.

```

#include <vector>
#include <filesystem>
#include "../Studentas.h"

```

Funkcijos

- `std::vector< std::string > nuskaitytiEilutesIVektoriu` (`const std::string &failas`)
Nuskaity visas tekstinio failo eilutes į string tipo vektorių.
- `std::vector< Studentas > nuskaitytiStudentuDuomenisIsFailo` (`const std::string &failas`)
Nuskaity studentų duomenis iš failo į vektorių.
- `void vykdytiNuskaitymąIsFailo ()`
Vykdo studentų duomenų nuskaitymą iš failo scenarijų.
- `void irasytiDuomenis` (`std::vector< Studentas > &studentuSarasas`)
Įrašo studentų duomenis pagal pasirinktą išvedimo scenarijų.
- `void irasytiStudentuDuomenisIsFaila` (`const std::vector< Studentas > &studentuSarasas, const std::string &failoPavadinimas`)
Įrašo studentų sąrašą į nurodytą failą.
- `void vykdytiSkirstymąIsFailu` (`std::vector< Studentas > &studentuSarasas`)
Vykdo studentų skirstymą į atskirus failus.
- `void irasytiSusikirstytusStudentusIsFailus` (`const std::vector< Studentas > &pazangiuSarasas, const std::vector< Studentas > &silpnuSarasas, const char &skaiciavimoMetodoPasirinkimas`)
Įrašo pažangius ir silpnus studentus į atskirus failus.
- `std::vector< std::filesystem::path > gautiTekstiniusFailus` (`const std::string &katalogas`)
Grąžina visus tekstinius failus iš nurodyto katalogo.
- `void praleistiTarpaisFailo` (`const char *&rodykle`)
Praleidžia tarpo simbolius skaitant tekstą iš simbolių rodyklės.
- `bool nuskaitytiZodisIsFailo` (`const char *&rodykle, std::string &isvestis`)
Nuskaity vieną žodį iš simbolių rodyklės.
- `bool nuskaitytiSveikaSkaiciusIsFailo` (`const char *&rodykle, int &x`)
Nuskaity sveikąjį skaičių iš simbolių rodyklės.
- `void nuskaitytiDuomenis` (`int pasirinkimasNuskaitymo, std::vector< Studentas > &studentuSarasas, const std::string &katalogas`)
Nuskaity studentų duomenis pagal pasirinktą failą iš katalogo.

5.5.1 Smulkus aprašymas

Funkcijos darbu su tekstiniais failais ir studentų duomenų nuskaitymu bei įrašymu.

5.5.2 Funkcijos Dokumentacija

5.5.2.1 gautiTekstiniusFailus()

```
std::vector< std::filesystem::path > gautiTekstiniusFailus (
    const std::string & katalogas)
```

Grąžina visus tekstinius failus iš nurodyto katalogo.

Parametrai

<i>katalogas</i>	Katalogo kelias.
------------------	------------------

Gražina

Tekstinių failų kelių vektorius.

5.5.2.2 irasytiDuomenis()

```
void irasytiDuomenis (
    std::vector< Studentas > & studentuSarasas)
```

Įrašo studentų duomenis pagal pasirinktą išvedimo scenarijų.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
------------------------	-------------------

5.5.2.3 irasytiStudentuDuomenisIFaila()

```
void irasytiStudentuDuomenisIFaila (
    const std::vector< Studentas > & studentuSarasas,
    const std::string & failoPavadinimas)
```

Įrašo studentų sąrašą į nurodytą failą.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
<i>failoPavadinimas</i>	Išvesties failo pavadinimas.

5.5.2.4 irasytiSuskirstytusStudentusIFailus()

```
void irasytiSuskirstytusStudentusIFailus (
    const std::vector< Studentas > & pazangiuSarasas,
    const std::vector< Studentas > & silpnuSarasas,
    const char & skaiciavimoMetodoPasirinkimas)
```

Įrašo pažangius ir silpnus studentus į atskirus failus.

Parametrai

<i>pazangiuSarasas</i>	Pažangių studentų sąrašas.
<i>silpnuSarasas</i>	Silpnų studentų sąrašas.
<i>skaiciavimoMetodoPasirinkimas</i>	Galutinio pažymio skaičiavimo metodas.

5.5.2.5 nuskaitytiDuomenis()

```
void nuskaitytiDuomenis (
    int pasirinkimasNuskaitymo,
    std::vector< Studentas > & studentuSarasas,
    const std::string & katalogas)
```

Nuskaityti studentų duomenis pagal pasirinktą failą iš katalogo.

Parametrai

<i>pasirinkimasNuskaitymo</i>	Pasirinkto failo numeris meniu sąraše.
-------------------------------	--

<i>studentuSarasas</i>	Studentų sąrašas, į kurį įrašomi nuskaityti duomenys.
<i>katalogas</i>	Katalogas, kuriame ieškoma tekstinių failų.

5.5.2.6 nuskaitytiEilutesIVektoriu()

```
std::vector< std::string > nuskaitytiEilutesIVektoriu (
    const std::string & failas)
```

Nuskaito visas tekstinio failo eilutes į string tipo vektorių.

Parametrai

<i>failas</i>	Failo kelias.
---------------	---------------

Gražina

Vektorius su nuskaitytomis eilutėmis.

5.5.2.7 nuskaitytiStudentuDuomenisIsFailo()

```
std::vector< Studentas > nuskaitytiStudentuDuomenisIsFailo (
    const std::string & failas)
```

Nuskaito studentų duomenis iš failo į vektorių.

Parametrai

<i>failas</i>	Failo kelias.
---------------	---------------

Gražina

Studentų vektorius.

5.5.2.8 nuskaitytiSveikaSkaiciulsFailo()

```
bool nuskaitytiSveikaSkaiciuIsFailo (
    const char *& rodykle,
    int & x)
```

Nuskaito sveikąjį skaičių iš simbolių rodyklės.

Parametrai

<i>rodykle</i>	Rodyklė į einamąją teksto poziciją.
----------------	-------------------------------------

<i>x</i>	Kintamasis, į kurį įrašomas nuskaitytas skaičius.
----------	---

Gražina

`true`, jei skaičius nuskaitytas sėkmingai, kitu atveju `false`.

5.5.2.9 nuskaitytiZodiIsFailo()

```
bool nuskaitytiZodiIsFailo (
    const char *& rodykle,
    std::string & isvestis)
```

Nuskaityti vieną žodį iš simbolių rodyklės.

Parametrai

<i>rodykle</i>	Rodyklė į einamąją teksto poziciją.
<i>isvestis</i>	Kintamasis, į kurį įrašomas nuskaitytas žodis.

Gražina

`true`, jei žodis nuskaitytas sėkmingai, kitu atveju `false`.

5.5.2.10 praleistiTarpaisFailo()

```
void praleistiTarpaisFailo (
    const char *& rodykle)
```

Praleidžia tarpo simbolius skaitant tekstą iš simbolių rodyklės.

Parametrai

<i>rodykle</i>	Rodyklė į einamąją teksto poziciją.
----------------	-------------------------------------

5.5.2.11 vykdytiSkirstymaIFailus()

```
void vykdytiSkirstymaIFailus (
    std::vector< Studentas > & studentuSarasas)
```

Vykdo studentų skirstymą į atskirus failus.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
------------------------	-------------------

5.6 strukturaDarbasSuFailais.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STRUKTURADARBASSUFILAI_H
00002 #define STRUKTURADARBASSUFILAI_H
00003
00004 #include <vector>
00005 #include <filesystem>
00006 #include "../Studentas.h"
00007
00012
00013 struct Failai;
00014
00020
00021 std::vector<std::string> nuskaitytiEilutesIVektoriu(const std::string& failas);
00022
00028 std::vector<Studentas> nuskaitytiStudentuDuomenisIsFailo(const std::string& failas);
00029
00033 void vykdytiNuskaitymaIsFailo();
00034
00039 void irasytiDuomenis(std::vector<Studentas>& studentuSarasas);
00040
00046 void irasytiStudentuDuomenisIFaila(const std::vector<Studentas>& studentuSarasas, const std::string&
    failoPavadinimas);
00047
00052 void vykdytiSkirstymaIFailus(std::vector<Studentas>& studentuSarasas);
00053
00060 void irasytiSuskirstytusStudentusIFailus(const std::vector<Studentas>& pazangiuSarasas, const
    std::vector<Studentas>& silpnuSarasas, const char& skaiciavimoMetodoPasirinkimas);
00061
00067 std::vector<std::filesystem::path> gautiTekstiniusFailus(const std::string& katalogas);
00068
00073 void praleistiTarpaIsFailo(const char*& rodykle);
00074
00081 bool nuskaitytiZodiIsFailo(const char*& rodykle, std::string& isvestis);
00082
00089 bool nuskaitytiSveikaSkaiciuIsFailo(const char*& rodykle, int& x);
00090
00097 void nuskaitytiDuomenis(int pasirinkimasNuskaitymo, std::vector<Studentas>& studentuSarasas, const
    std::string& katalogas);
00098
00099 #endif
```

5.7 strukturosFailai/strukturaGeneravimas.h Failo Nuoroda

Funkcijos studentų vardams, pavardėms ir pažymiams generuoti.

```
#include <vector>
#include "../Studentas.h"
#include "../strukturosFailai/Failai.h"
```

Funkcijos

- std::pair< std::vector< int >, int > [generuotiRezultatus](#) (int maksimalusNDKiekis)
Sugeneruoja namų darbų pažymius ir egzamino rezultata.
- std::pair< std::string, std::string > [generuotiVardaPavarde](#) (const std::vector< std::string > &vyrVardai, const std::vector< std::string > &vyrPavardes, const std::vector< std::string > &motVardai, const std::vector< std::string > &motPavardes)
Sugeneruoja atsitiktinį vardą ir pavardę.
- int [generuotiSveikaSkaiciu](#) (int nuo, int iki)
Sugeneruoja atsitiktinį sveikąjį skaičių nurodytame intervale.
- std::vector< [Studentas](#) > [generuotiStudentus](#) (int studentuKiekis, int maksimalusNDKiekis, const [Failai](#) &failai)
Sugeneruoja nurodytą kiekį studentų.

5.7.1 Smulkus aprašymas

Funkcijos studentų vardams, pavardėms ir pažymiams generuoti.

5.7.2 Funkcijos Dokumentacija

5.7.2.1 generuotiRezultatus()

```
std::pair< std::vector< int >, int > generuotiRezultatus (
    int maksimalusNDKiekis)
```

Sugeneruoja namų darbų pažymius ir egzamino rezultatą.

Parametrai

<i>maksimalusNDKiekis</i>	Maksimalus namų darbų kiekis.
---------------------------	-------------------------------

Gražina

Pora: namų darbų pažymių vektorius ir egzamino pažymys.

5.7.2.2 generuotiStudentus()

```
std::vector< Studentas > generuotiStudentus (
    int studentuKiekis,
    int maksimalusNDKiekis,
    const Failai & failai)
```

Sugeneruoja nurodytą kiekį studentų.

Parametrai

<i>studentuKiekis</i>	Studentų kiekis.
<i>maksimalusNDKiekis</i>	Maksimalus namų darbų kiekis.
<i>failai</i>	Struktūra su vardų ir pavardžių sąrašais.

Gražina

Sugeneruotų studentų vektorius.

5.7.2.3 generuotiSveikaSkaiciu()

```
int generuotiSveikaSkaiciu (
    int nuo,
    int iki)
```

Sugeneruoja atsitiktinį sveikąjį skaičių nurodytame intervale.

Parametrai

<i>nuo</i>	Apatinė intervalo riba.
------------	-------------------------

<i>iki</i>	Viršutinė intervalo riba.
------------	---------------------------

Gražina

Atsitiktinis sveikasis skaičius iš intervalo [nuo; iki].

5.7.2.4 generuotiVardaPavarde()

```
std::pair< std::string, std::string > generuotiVardaPavarde (
    const std::vector< std::string > & vyrVardai,
    const std::vector< std::string > & vyrPavardes,
    const std::vector< std::string > & motVardai,
    const std::vector< std::string > & motPavardes)
```

Sugeneruoja atsitiktinį vardą ir pavardę.

Parametrai

<i>vyrVardai</i>	Vyrų vardų sąrašas.
<i>vyrPavardes</i>	Vyrų pavardžių sąrašas.
<i>motVardai</i>	Moterų vardų sąrašas.
<i>motPavardes</i>	Moterų pavardžių sąrašas.

Gražina

Pora: sugeneruotas vardas ir pavardė.

5.8 strukturaGeneravimas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STRUKTURAGENERAVIMAS_H
00002 #define STRUKTURAGENERAVIMAS_H
00003
00004 #include <vector>
00005 #include "../Studentas.h"
00006 #include "../strukturosFailai/Failai.h"
00007
00012
00018 std::pair<std::vector<int>, int> generuotiRezultatus(int maksimalusNDKiekis);
00019
00028 std::pair<std::string, std::string> generuotiVardaPavarde(const std::vector<std::string>& vyrVardai,
    const std::vector<std::string>& vyrPavardes, const std::vector<std::string>& motVardai, const
    std::vector<std::string>& motPavardes);
00029
00036 int generuotiSveikaSkaiciu(int nuo, int iki);
00037
00045 std::vector<Studentas> generuotiStudentus(int studentuKiekis, int maksimalusNDKiekis, const Failai&
    failai);
00046
00047 #endif
```

5.9 struktūrosFailai/strukturalSvestis.h Failo Nuoroda

Funkcijos studentų duomenų ir testavimo rezultatų išvedimui.

```
#include <vector>
#include <string>
#include <ostream>
#include "../Studentas.h"
#include "../struktūrosFailai/strukturalSvestis.h"
```

Funkcijos

- void [parodytiRezultatuLentele](#) (std::ostream &out, const std::vector< [Studentas](#) > &studentuSarasas, char skaiciavimoMetodoPasirinkimas)
Parodo studentų rezultatų lentelę nurodytame išvesties sraute.
- void [isvestiStudentus](#) (int pasirinkimasIšvedimo, const std::vector< [Studentas](#) > &studentuSarasas, char skaiciavimoMetodoPasirinkimas)
Išveda studentų sąrašą pagal pasirinktą išvedimo būdą.
- void [apdorotiIrsvestiStudentus](#) (std::vector< [Studentas](#) > &studentuSarasas, char skaiciavimoMetodoPasirinkimas)
Apdoroja studentų sąrašą ir išveda rezultatus.
- void [parodytiStudentus](#) (const std::vector< [Studentas](#) > &studentai, char skaiciavimoMetodas)
Išveda studentus į ekraną.
- void [spausdintiVidurkius](#) (const [TestoLaikai](#) &laikai)
Spausdina testavimo laikų vidurkius.
- void [vykdytilrasymaIFaila](#) ([Failai](#) &failai)
Vykdo studentų duomenų generavimą ir įrašymą į failą.
- void [irasytiStudentuDuomenisIFaila](#) (std::vector< [Studentas](#) > &studentuSarasas, int maksimalusNDKiekis, int studentuKiekis, [Failai](#) &failai)
Įrašo sugeneruotus studentų duomenis į failą.

5.9.1 Smulkus aprašymas

Funkcijos studentų duomenų ir testavimo rezultatų išvedimui.

5.9.2 Funkcijos Dokumentacija

5.9.2.1 apdorotiIrsvestiStudentus()

```
void apdorotiIrsvestiStudentus (
    std::vector< Studentas > & studentuSarasas,
    char skaiciavimoMetodoPasirinkimas)
```

Apdoroja studentų sąrašą ir išveda rezultatus.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
------------------------	-------------------

<i>skaiciavimoMetodoPasirinkimas</i>	Galutinio pažymio skaičiavimo metodas.
--------------------------------------	--

5.9.2.2 irasytiStudentuDuomenisIFaila()

```
void irasytiStudentuDuomenisIFaila (
    std::vector< Studentas > & studentuSarasas,
    int maksimalusNDKiekis,
    int studentuKiekis,
    Failai & failai)
```

Jrašo sugeneruotus studentų duomenis į failą.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
<i>maksimalusNDKiekis</i>	Maksimalus namų darbų kiekis.
<i>studentuKiekis</i>	Studentų kiekis.
<i>failai</i>	Vardų ir pavardžių failų duomenų struktūra.

5.9.2.3 isvestiStudentus()

```
void isvestiStudentus (
    int pasirinkimasIsvedimo,
    const std::vector< Studentas > & studentuSarasas,
    char skaiciavimoMetodoPasirinkimas)
```

Išveda studentų sąrašą pagal pasirinktą išvedimo būdą.

Parametrai

<i>pasirinkimasIsvedimo</i>	Išvedimo būdo numeris.
<i>studentuSarasas</i>	Studentų sąrašas.
<i>skaiciavimoMetodoPasirinkimas</i>	Galutinio pažymio skaičiavimo metodas.

5.9.2.4 parodytiRezultatuLentele()

```
void parodytiRezultatuLentele (
    std::ostream & out,
    const std::vector< Studentas > & studentuSarasas,
    char skaiciavimoMetodoPasirinkimas)
```

Parodo studentų rezultatų lentelę nurodytame išvesties sraute.

Parametrai

<i>out</i>	Išvesties srautas.
------------	--------------------

<i>studentuSarasas</i>	Studentų sąrašas.
<i>skaiciavimoMetodoPasirinkimas</i>	Galutinio pažymio skaičiavimo metodas.

5.9.2.5 parodytiStudentus()

```
void parodytiStudentus (
    const std::vector< Studentas > & studentai,
    char skaiciavimoMetodas)
```

Išveda studentus į ekraną.

Parametrai

<i>studentai</i>	Studentų sąrašas.
<i>skaiciavimoMetodas</i>	Galutinio pažymio skaičiavimo metodas.

5.9.2.6 spausdintiVidurkius()

```
void spausdintiVidurkius (
    const TestoLaikai & laikai)
```

Spausdina testavimo laikų vidurkius.

Parametrai

<i>laikai</i>	Testavimo laikų struktūra.
---------------	----------------------------

5.9.2.7 vykdytilrasymaIFaila()

```
void vykdytiIrasymaIFaila (
    Failai & failai)
```

Vykdo studentų duomenų generavimą ir įrašymą į failą.

Parametrai

<i>failai</i>	Vardų ir pavardžių failų duomenų struktūra.
---------------	---

5.10 strukturalvestis.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STRUKTURALIVESTIS_H
00002 #define STRUKTURALIVESTIS_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <ostream>
00007 #include "../Studentas.h"
00008 #include "../strukturosFailai/strukturaTestavimas.h"
00009
00014
00021 void parodytiRezultatuLentele(std::ostream& out, const std::vector<Studentas>& studentuSarasas, char
skaiciavimoMetodoPasirinkimas);
00022
00029 void isvestiStudentus(int pasirinkimasIsvedimo, const std::vector<Studentas>& studentuSarasas, char
skaiciavimoMetodoPasirinkimas);
00030
00036 void apdorotiIrIsvestiStudentus(std::vector<Studentas>& studentuSarasas, char
skaiciavimoMetodoPasirinkimas);
00037
00043 void parodytiStudentus(const std::vector<Studentas>& studentai, char skaiciavimoMetodas);
00044
00049 void spausdintiVidurkcius(const TestoLaikai& laikai);
00050
00055 void vykdytiIrasymaIFaila(Failai& failai);
00056
00064 void irasytiStudentuDuomenisIFaila(std::vector<Studentas>& studentuSarasas, int maksimalusNDKiekis,
int studentuKiekis, Failai& failai);
00065
00066 #endif
```

5.11 strukturosFailai/strukturalvestis.h Failo Nuoroda

Funkcijos naudotojo įvesties nuskaitymui, tikrinimui ir apdorojimui.

```
#include <vector>
#include <string>
#include "../Studentas.h"
#include "../strukturosFailai/Failai.h"
```

Funkcijos

- void [nuskaitytiNamuDarbuPazymius](#) (std::vector< int > &namuDarbuPazymiai, int maksimalusNDKiekis)
Nuskaityti namų darbų pažymius iš naudotojo.
- int [nuskaitytiMenuPasirinkima](#) (const std::vector< std::string > &eilutes)
Nuskaityti meniu pasirinkimą iš tekstinių eilučių sąrašo.
- bool [nuskaitytiPagrindinioMenuPasirinkima](#) (const std::vector< std::string > &eilutes, int &pasirinkimas)
Nuskaityti pagrindinio meniu pasirinkimą.
- int [nuskaitytiNeneigiamąSveikąSkaiciu](#) (const char *pranesimas)
Nuskaityti neneigiamą sveikąjį skaičių.
- int [nuskaitytiTeigiamąSveikąSkaiciu](#) (const char *pranesimas)
Nuskaityti teigiamą sveikąjį skaičių.
- char [nuskaitytiSkaiciavimoMetoda](#) ()
Nuskaityti galutinio pažymio skaičiavimo metodą.
- int [nuskaitytiPazymiNuo1iki10](#) (const char *pranesimas)
Nuskaityti pažymį nuo 1 iki 10.
- void [nuskaitytiSveikąSkaiciu](#) (const std::string &ivestis, int &reiksme)
Konvertuoja tekstinę įvestį į sveikąjį skaičių.

- void `tikrintilvesti` (const std::string &ivestis)
Patikrina, ar įvestis yra korektiška.
- std::string `nuskaitytiVardaArPavarde` (const char *ivestiesPranesimas, void(*tvarkyti)(std::string &), const char *klaidosPranesimas)
Nuskaityti vardą arba pavardę ir pritaiko nurodytą tvarkymo funkciją.
- bool `patvirtintiNaujoStudentoPridejima` ()
Paklausia naudotojo, ar pridėti dar vieną studentą.
- void `vykdytiStudentulvedima` (bool generuotiPazymius)
Vykdo studentų įvedimo procesą.
- void `vykdytiPilnaGeneravima` (Failai &failai)
Vykdo pilną studentų generavimo scenarijų.
- void `tvarkytiVarda` (std::string &ivestis)
Sutvarko vardo įvestį.
- void `tvarkytiPavarde` (std::string &ivestis)
Sutvarko pavardės įvestį.
- std::string `saugiaiNuskaitytiEilute` ()
Saugiai nuskaityti vieną eilutę iš standartinės įvesties.
- void `parinktiRikiavimoBudus` (int &pasirinkimasRikiavimoPazangiu, int &pasirinkimasRikiavimoSilpnu)
Parenta pažangių ir silpnų studentų rikiavimo būdus.
- std::string `suformuotiStudentolvestiesEilute` (const std::string &vardas, const std::string &pavarde, const std::vector< int > &namuDarbai, int egzaminoPazymys)
Suformuoja studento įvesties eilutę iš atskirų laukų.
- `Studentas sukurtiStudentalsEilutesPerOperatoriu` (const std::string &eilute)
Sukuria studentą iš tekstinės eilutės naudojant įvesties operatorių.

5.11.1 Smulkus aprašymas

Funkcijos naudotojo įvesties nuskaitymui, tikrinimui ir apdorojimui.

5.11.2 Funkcijos Dokumentacija

5.11.2.1 nuskaitytiMenuPasirinkima()

```
int nuskaitytiMenuPasirinkima (
    const std::vector< std::string > & eilutes) [inline]
```

Nuskaityti menu pasirinkimą iš tekstinių eilučių sąrašo.

Nuskaityti pasirinkimą iš dinaminio menu eilučių vektoriaus.

Parametrai

<code>eilutes</code>	Menu eilučių vektorius.
----------------------	-------------------------

Gražina

Pasirinktas meniu punktas.

Nuskaityto meniu pasirinkimą iš tekstinių eilučių sąrašo.

Parametrai

<i>eilutes</i>	Meniu eilučių vektorius.
----------------	--------------------------

Gražina

Pasirinkto meniu punkto numeris.

5.11.2.2 nuskaitytiNamuDarbuPazymius()

```
void nuskaitytiNamuDarbuPazymius (
    std::vector< int > & namuDarbuPazymiai,
    int maksimalusNDKiekis)
```

Nuskaityto namų darbų pažymius iš naudotojo.

Parametrai

<i>namuDarbuPazymiai</i>	Vektorius, į kurį įrašomi pažymiai.
<i>maksimalusNDKiekis</i>	Maksimalus leidžiamas namų darbų kiekis.

5.11.2.3 nuskaitytiNeneigiamaSveikajiSkaiciu()

```
int nuskaitytiNeneigiamaSveikajiSkaiciu (
    const char * pranesimas)
```

Nuskaityto neneigiamą sveikąjį skaičių.

Parametrai

<i>pranesimas</i>	Įvesties pranešimas naudotojui.
-------------------	---------------------------------

Gražina

Neneigiamas sveikasis skaičius.

5.11.2.4 nuskaitytiPagrindinioMenuPasirinkima()

```
bool nuskaitytiPagrindinioMenuPasirinkima (
    const std::vector< std::string > & eilutes,
    int & pasirinkimas)
```

Nuskaityto pagrindinio meniu pasirinkimą.

Parametrai

<i>eilutes</i>	Meniu eilučių vektorius.
----------------	--------------------------

<i>pasirinkimas</i>	Kintamasis, į kurį įrašomas pasirinkimas.
---------------------	---

Gražina

`true`, jei pasirinkimas nuskaitytas sėkmingai.

5.11.2.5 nuskaitytiPazymiNuo1iki10()

```
int nuskaitytiPazymiNuo1iki10 (
    const char * pranesimas)
```

Nuskaito pažymį nuo 1 iki 10.

Parametrai

<i>pranesimas</i>	Įvesties pranešimas naudotojui.
-------------------	---------------------------------

Gražina

Pažymys intervale [1; 10].

5.11.2.6 nuskaitytiSkaiciavimoMetoda()

```
char nuskaitytiSkaiciavimoMetoda ()
```

Nuskaito galutinio pažymio skaičiavimo metodą.

Gražina

Skaičiavimo metodo simbolis.

5.11.2.7 nuskaitytiSveikajiSkaiciu()

```
void nuskaitytiSveikajiSkaiciu (
    const std::string & investis,
    int & reiksme)
```

Konvertuoja tekstinę įvestį į sveikąjį skaičių.

Parametrai

<i>investis</i>	Tekstinė įvestis.
<i>reiksme</i>	Kintamasis, į kurį įrašoma konvertuota reikšmė.

5.11.2.8 nuskaitytiTeigiamaSveikajiSkaiciu()

```
int nuskaitytiTeigiamaSveikajiSkaiciu (
    const char * pranesimas)
```

Nuskaityto teigiamą sveikąjį skaičių.

Parametrai

<i>pranesimas</i>	Įvesties pranešimas naudotojui.
-------------------	---------------------------------

Gražina

Teigiamas sveikasis skaičius.

5.11.2.9 nuskaitytiVardaArPavarde()

```
std::string nuskaitytiVardaArPavarde (
    const char * ivestiesPranesimas,
    void(* tvarkyti ) (std::string &),
    const char * klaidosPranesimas)
```

Nuskaityto vardą arba pavardę ir pritaiko nurodytą tvarkymo funkciją.

Parametrai

<i>ivestiesPranesimas</i>	Pranešimas naudotojui.
<i>tvarkyti</i>	Funkcija vardui arba pavardei tvarkyti.
<i>klaidosPranesimas</i>	Klaidos pranešimas neteisingos įvesties atveju.

Gražina

Sutvarkytas vardas arba pavardė.

5.11.2.10 parinktiRikiavimoBudus()

```
void parinktiRikiavimoBudus (
    int & pasirinkimasRikiavimoPazangiu,
    int & pasirinkimasRikiavimoSilpnu)
```

Parenka pažangių ir silpnų studentų rikiavimo būdus.

Parametrai

<i>pasirinkimasRikiavimoPazangiu</i>	Pažangių studentų rikiavimo pasirinkimas.
<i>pasirinkimasRikiavimoSilpnu</i>	Silpnų studentų rikiavimo pasirinkimas.

5.11.2.11 patvirtintiNaujoStudentoPridejima()

```
bool patvirtintiNaujoStudentoPridejima ()
```

Paklausia naudotojo, ar pridėti dar vieną studentą.

Gražina

true, jei naudotojas patvirtina naujo studento pridėjimą.

5.11.2.12 saugiaiNuskaitytiEilute()

```
std::string saugiaiNuskaitytiEilute ()
```

Saugiai nuskaityti vieną eilutę iš standartinės įvesties.

Gražina

Nuskaityta eilutė.

5.11.2.13 suformuotiStudentoIvestiesEilute()

```
std::string suformuotiStudentoIvestiesEilute (  
    const std::string & vardas,  
    const std::string & pavarde,  
    const std::vector< int > & namuDarbai,  
    int egzaminoPazymys)
```

Suformuoja studento įvesties eilutę iš atskirų laukų.

Parametrai

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavardė.
<i>namuDarbai</i>	Namų darbų pažymių vektorius.
<i>egzaminoPazymys</i>	Egzamino pažymys.

Gražina

Suformuota studento duomenų eilutė.

5.11.2.14 sukurtiStudentaisEilutesPerOperatoriu()

```
Studentas sukurtiStudentaisEilutesPerOperatoriu (  
    const std::string & eilute)
```

Sukuria studentą iš tekstinės eilutės naudojant įvesties operatorių.

Parametrai

<i>eilute</i>	Studento duomenų eilutė.
---------------	--------------------------

Gražina

Sukurtas [Studentas](#) objektas.

5.11.2.15 tikrintiIvesti()

```
void tikrintiIvesti (  
    const std::string & iverstis)
```

Patikrina, ar įvestis yra korektiška.

Parametrai

<i>iverstis</i>	Tikrinama tekstinė įvestis.
-----------------	-----------------------------

Išimtys

<i>std::exception</i>	Jei įvestis nekorektiška.
-----------------------	---------------------------

5.11.2.16 tvarkytiPavarde()

```
void tvarkytiPavarde (  
    std::string & iverstis)
```

Sutvarko pavardės įvestį.

Parametrai

<i>iverstis</i>	Tvarkoma pavardės eilutė.
-----------------	---------------------------

5.11.2.17 tvarkytiVarda()

```
void tvarkytiVarda (  
    std::string & iverstis)
```

Sutvarko vardo įvestį.

Parametrai

<i>iverstis</i>	Tvarkoma vardo eilutė.
-----------------	------------------------

5.11.2.18 vykdytiPilnaGeneravima()

```
void vykdytiPilnaGeneravima (
    Failai & failai)
```

Vykdo pilną studentų generavimo scenarijų.

Parametrai

<i>failai</i>	Vardų ir pavardžių failų duomenys.
---------------	------------------------------------

5.11.2.19 vykdytiStudentuIvedima()

```
void vykdytiStudentuIvedima (
    bool generuotiPazymius)
```

Vykdo studentų įvedimo procesą.

Parametrai

<i>generuotiPazymius</i>	Nurodo, ar pažymius generuoti automatiškai.
--------------------------	---

5.12 strukturalvestis.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STRUKTURAIVESTIS_H
00002 #define STRUKTURAIVESTIS_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include "../Studentas.h"
00007 #include "../strukturosFailai/Failai.h"
00008
00013
00019 void nuskaitytiNamuDarbuPazymius(std::vector<int>& namuDarbuPazymiai, int maksimalusNDKiekis);
00020
00026 int nuskaitytiMeniuPasirinkima(const std::vector<std::string>& eilutes);
00027
00034 bool nuskaitytiPagrindinioMeniuPasirinkima(const std::vector<std::string>& eilutes, int&
    pasirinkimas);
00035
00041 int nuskaitytiNeneigiamaSveikajiSkaiciu(const char* pranesimas);
00042
00048 int nuskaitytiTeigiamaSveikajiSkaiciu(const char* pranesimas);
00049
00054 char nuskaitytiSkaiciavimoMetoda();
00055
00061 int nuskaitytiPazymiNuoliki10(const char* pranesimas);
00062
00068 void nuskaitytiSveikajiSkaiciu(const std::string& iverstis, int& reiksme);
00069
00075 void tikrintiIvesti(const std::string& iverstis);
00076
00084 std::string nuskaitytiVardaArPavarde(
00085     const char* iverstiesPranesimas,
00086     void(*tvarkyti)(std::string&),
00087     const char* klaidosPranesimas
00088 );
00089
00094 bool patvirtintiNaujoStudentoPridejima();
00095
00100 void vykdytiStudentuIvedima(bool generuotiPazymius);
00101
00106 void vykdytiPilnaGeneravima(Failai& failai);
00107
```

```

00112 void tvarkytiVarda(std::string& ivestis);
00113
00118 void tvarkytiPavarde(std::string& ivestis);
00119
00124 std::string saugiaiNuskaitytiEilute();
00125
00131 void parinktiRikiavimoBudus(
00132     int& pasirinkimasRikiavimoPazangiu,
00133     int& pasirinkimasRikiavimoSilpnu
00134 );
00135
00144 std::string suformuotiStudentoIvestiesEilute(
00145     const std::string& vardas,
00146     const std::string& pavarde,
00147     const std::vector<int>& namuDarbai,
00148     int egzaminoPazymys
00149 );
00150
00156 Studentas sukurtiStudentaIsEilutesPerOperatoriu(const std::string& eilute);
00157
00158 #endif

```

5.13 struktūrosFailai/strukturaMenu.h Failo Nuoroda

Programos meniu konstantos ir meniu pasirinkimų nuskaitymo funkcijos.

```

#include <array>
#include <string_view>
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
#include "../struktūrosFailai/strukturaIvestis.h"

```

Tipų apibrėžimai

- using **MenuEilute** = std::string_view
Menu eilutės tipas.

Funkcijos

- template<std::size_t N>
int **nuskaitytiMenuPasirinkima** (const std::array< std::string_view, N > &eilutes)
Nuskaityti pasirinkimą iš statinio meniu masyvo.
- std::vector< std::string > **gautiNuskaitymoMenu** (const std::string &katalogas)
Sugeneruoja failų nuskaitymo meniu pagal kataloge esančius tekstinius failus.
- template<std::size_t N>
bool **nuskaitytiPagrindinioMenuPasirinkima** (const std::array< std::string_view, N > &eilutes, int &pasirinkimas)
Nuskaityti pagrindinio meniu pasirinkimą iš statinio meniu masyvo.
- int **nuskaitytiMenuPasirinkima** (const std::vector< std::string > &eilutes)
Nuskaityti pasirinkimą iš dinaminio meniu eilučių vektoriaus.

Kintamieji

- `const std::array< MeniuEilute, 15 > PAGRINDINIS_MENIU`
Pagrindinio programos meniu eilutės.
- `const std::array< MeniuEilute, 7 > RIKIAVIMO_MENIU`
Bendro rikiavimo meniu eilutės.
- `const std::array< MeniuEilute, 4 > RIKIAVIMO_MENIU_TIK_DIDEJANCIAI`
Rikiavimo tik didėjančia tvarka meniu eilutės.
- `const std::array< MeniuEilute, 7 > PAZANGIU_RIKIAVIMO_MENIU`
Pažangių studentų rikiavimo meniu eilutės.
- `const std::array< MeniuEilute, 7 > SILPNU_RIKIAVIMO_MENIU`
Silpnų studentų rikiavimo meniu eilutės.
- `const std::array< MeniuEilute, 3 > ISVEDIMO_MENIU`
Išvedimo būdų meniu eilutės.
- `const std::array< MeniuEilute, 6 > ISVEDIMO_I_FAILA_MENIU`
Išvedimo į failą meniu eilutės.

5.13.1 Smulkus aprašymas

Programos meniu konstantos ir meniu pasirinkimų nuskaitymo funkcijos.

5.13.2 Funkcijos Dokumentacija

5.13.2.1 gautiNuskaitymoMenu()

```
std::vector< std::string > gautiNuskaitymoMenu (
    const std::string & katalogas)
```

Sugeneruoja failų nuskaitymo meniu pagal kataloge esančius tekstinius failus.

Parametrai

<i>katalogas</i>	Katalogas, kuriame ieškoma tekstinių failų.
------------------	---

Gražina

Menu eilučių vektorius.

5.13.2.2 nuskaitytiMenuPasirinkima() [1/2]

```
template<std::size_t N>
int nuskaitytiMenuPasirinkima (
    const std::array< std::string_view, N > & eilutes)
```

Nuskaityti pasirinkimą iš statinio meniu masyvo.

Funkcija kartoją įvestį tol, kol naudotojas įveda korektišką pasirinkimą.

Template Parameters

<i>N</i>	Menu eilučių kiekis.
----------	----------------------

Parametrai

<i>eilutes</i>	Menu eilučių masyvas.
----------------	-----------------------

Gražina

Pasirinkto meniu punkto numeris.

5.13.2.3 nuskaitytiMenuPasirinkima() [2/2]

```
int nuskaitytiMenuPasirinkima (
    const std::vector< std::string > & eilutes) [inline]
```

Nuskaityti pasirinkimą iš dinaminio meniu eilučių vektoriaus.

Nuskaityti meniu pasirinkimą iš tekstinių eilučių sąrašo.

Parametrai

<i>eilutes</i>	Menu eilučių vektorius.
----------------	-------------------------

Gražina

Pasirinkto meniu punkto numeris.

Nuskaityti pasirinkimą iš dinaminio meniu eilučių vektoriaus.

Parametrai

<i>eilutes</i>	Menu eilučių vektorius.
----------------	-------------------------

Gražina

Pasirinktas meniu punktas.

5.13.2.4 nuskaitytiPagrindinioMenuPasirinkima()

```
template<std::size_t N>
bool nuskaitytiPagrindinioMenuPasirinkima (
    const std::array< std::string_view, N > & eilutes,
    int & pasirinkimas)
```

Nuskaityti pagrindinio meniu pasirinkimą iš statinio meniu masyvo.

Funkcija kartoja įvestį tol, kol naudotojas įveda korektišką pasirinkimą.

Template Parameters

<i>N</i>	Menu eilučių kiekis.
----------	----------------------

Parametrai

<i>eilutes</i>	Menu eilučių masyvas.
<i>pasirinkimas</i>	Kintamasis, į kurį įrašomas pasirinkimas.

Gražina

true, jei pasirinkimas nuskaitytas sėkmingai.

5.14 strukturaMenu.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STRUKTURAMENIU_H
00002 #define STRUKTURAMENIU_H
00003
00004 #include <array>
00005 #include <string_view>
00006 #include <iostream>
00007 #include <string>
00008 #include <vector>
00009 #include <stdexcept>
00010 #include "../strukturosFailai/strukturaIvestis.h"
00011
00016
00018 using MenuEilute = std::string_view;
00019
00021 extern const std::array<MenuEilute, 15> PAGRINDINIS_MENU;
00022
00024 extern const std::array<MenuEilute, 7> RIKIAVIMO_MENU;
00025
00027 extern const std::array<MenuEilute, 4> RIKIAVIMO_MENU_TIK_DIDEJANCIAI;
00028
00030 extern const std::array<MenuEilute, 7> PAZANGIU_RIKIAVIMO_MENU;
00031
00033 extern const std::array<MenuEilute, 7> SILPNU_RIKIAVIMO_MENU;
00034
00036 extern const std::array<MenuEilute, 3> ISVEDIMO_MENU;
00037
00039 extern const std::array<MenuEilute, 6> ISVEDIMO_I_FAILA_MENU;
00040
00050 template <std::size_t N>
00051 int nuskaitytiMenuPasirinkima(const std::array<std::string_view, N>& eilutes) {
00052     while (true) {
00053         try {
00054             std::cout << std::string(98, '-') << "\n";
00055             for (const auto& eilute : eilutes) std::cout << eilute << "\n";
00056             std::cout << std::string(98, '-') << "\n";
00057             std::cout << "Pasirinkite programos eiga: ";
```

```

00058         std::string iverstis = saugiaiNuskaitytiEilute();
00059         int menu = 0;
00060         const int maxMenu = static_cast<int>(eilutes.size()) - 1;
00061         tikrintiIvesti(iverstis);
00062         nuskaitytiSveikajiSkaiciu(iverstis, menu);
00063         if (menu < 1 || menu > maxMenu) throw std::out_of_range("Pasirinkimas turi būti nuo 1
iki " + std::to_string(maxMenu) + ".");
00064         return menu;
00065     } catch (const std::exception& e) {
00066         std::cout << "Klaida: " << e.what() << "\n";
00067     }
00068 }
00069 }
00070
00076 std::vector<std::string> gautiNuskaitymoMenu(const std::string& katalogas);
00077
00088 template <std::size_t N>
00089 bool nuskaitytiPagrindinioMenuPasirinkima(const std::array<std::string_view, N>& eilutes, int&
pasirinkimas) {
00090     const int didziausiasPasirinkimas = static_cast<int>(N) - 1;
00091     while (true) {
00092         try {
00093             std::cout << std::string(98, '-') << '\n';
00094             for (const auto& eilute : eilutes) {
00095                 std::cout << eilute << '\n';
00096             }
00097             std::cout << std::string(98, '-') << '\n';
00098             std::cout << "Pasirinkite programos eiga: ";
00099             const std::string iverstis = saugiaiNuskaitytiEilute();
00100             tikrintiIvesti(iverstis);
00101             nuskaitytiSveikajiSkaiciu(iverstis, pasirinkimas);
00102             if (pasirinkimas < 1 || pasirinkimas > didziausiasPasirinkimas) throw std::out_of_range(
"Pasirinkimas turi būti nuo 1 iki " + std::to_string(didziausiasPasirinkimas) + ".");
00103             return true;
00104         } catch (const std::exception& klaida) {
00105             std::cout << "Klaida: " << klaida.what() << '\n';
00106         }
00107     }
00108 }
00109
00115 inline int nuskaitytiMenuPasirinkima(const std::vector<std::string>& eilutes) {
00116     while (true) {
00117         try {
00118             std::cout << std::string(98, '-') << "\n";
00119             for (const auto& eilute : eilutes) { std::cout << eilute << "\n";}
00120             std::cout << std::string(98, '-') << "\n";
00121             std::cout << "Pasirinkite programos eiga: ";
00122             std::string iverstis = saugiaiNuskaitytiEilute();
00123             int menu = 0;
00124             const int maxMenu = static_cast<int>(eilutes.size()) - 1;
00125             tikrintiIvesti(iverstis);
00126             nuskaitytiSveikajiSkaiciu(iverstis, menu);
00127             if (menu < 1 || menu > maxMenu) throw std::out_of_range("Pasirinkimas turi būti nuo 1
iki " + std::to_string(maxMenu) + ".");
00128             return menu;
00129         } catch (const std::exception& e) {
00130             std::cout << "Klaida: " << e.what() << "\n";
00131         }
00132     }
00133 }
00134
00135 #endif

```

5.15 struktūrosFailai/strukturaRikiavimas.h Failo Nuoroda

Funkcijos studentų sąrašams rikiuoti.

```

#include <vector>
#include "../Studentas.h"

```

Funkcijos

- void [rikiuotiStudentus](#) (int pasirinkimasRikiavimo, std::vector< [Studentas](#) > &studentuSarasas)
Rikiuoja studentų sąrašą pagal pasirinktą kriterijų.

- void `rikiuotiSuskirstytusStudentus` (`std::vector< Studentas > &pazangiuSarasas`, `std::vector< Studentas > &silpnuSarasas`, `int pasirinkimasRikiavimoPazangiu`, `int pasirinkimasRikiavimoSilpnu`)

Rikiuoja pažangių ir silpnų studentų sąrašus pagal pasirinktus kriterijus.

5.15.1 Smulkus aprašymas

Funkcijos studentų sąrašams rikiuoti.

5.15.2 Funkcijos Dokumentacija

5.15.2.1 rikiuotiStudentus()

```
void rikiuotiStudentus (
    int pasirinkimasRikiavimo,
    std::vector< Studentas > & studentuSarasas)
```

Rikiuoja studentų sąrašą pagal pasirinktą kriterijų.

Tipiniai pasirinkimai:

- pagal vardą;
- pagal pavardę;
- pagal galutinį pažymį.

Parametrai

<code>pasirinkimasRikiavimo</code>	Rikiavimo kriterijaus numeris.
<code>studentuSarasas</code>	Studentų sąrašas, kuris bus rikiuojamas.

5.15.2.2 rikiuotiSuskirstytusStudentus()

```
void rikiuotiSuskirstytusStudentus (
    std::vector< Studentas > & pazangiuSarasas,
    std::vector< Studentas > & silpnuSarasas,
    int pasirinkimasRikiavimoPazangiu,
    int pasirinkimasRikiavimoSilpnu)
```

Rikiuoja pažangių ir silpnų studentų sąrašus pagal pasirinktus kriterijus.

Parametrai

<code>pazangiuSarasas</code>	Pažangių studentų sąrašas.
<code>silpnuSarasas</code>	Silpnų studentų sąrašas.
<code>pasirinkimasRikiavimoPazangiu</code>	Pažangių studentų rikiavimo kriterijus.
<code>pasirinkimasRikiavimoSilpnu</code>	Silpnų studentų rikiavimo kriterijus.

5.16 strukturaRikiavimas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STRUKTURARIKIAVIMAS_H
00002 #define STRUKTURARIKIAVIMAS_H
00003
00004 #include <vector>
00005 #include "../Studentas.h"
00006
00011
00023 void rikiuotiStudentus(int pasirinkimasRikiavimo, std::vector<Studentas>& studentuSarasas);
00031 void rikiuotiSuskirstytusStudentus(std::vector<Studentas>& pazangiuSarasas, std::vector<Studentas>&
    silpnuSarasas, int pasirinkimasRikiavimoPazangiu, int pasirinkimasRikiavimoSilpnu);
00032
00033 #endif
```

5.17 strukturosFailai/strukturaSkaiciavimai.h Failo Nuoroda

Funkcijos studentų pažymių skaičiavimui ir skirstymui.

```
#include <vector>
#include "../Studentas.h"
```

Funkcijos

- double [skaiciuotiNDVidurki](#) (const std::vector< int > &ndPazymiai)
Apskaiciuoja namų darbų pažymių vidurkį.
- double [skaiciuotiGalutiniVidurki](#) (const [Studentas](#) &studentas)
Apskaiciuoja galutinį pažymį pagal namų darbų vidurkį.
- double [skaiciuotiNDMediana](#) (std::vector< int > ndPazymiai)
Apskaiciuoja namų darbų pažymių medianą.
- double [skaiciuotiGalutineMediana](#) (const [Studentas](#) &studentas)
Apskaiciuoja galutinį pažymį pagal namų darbų medianą.
- template<typename T, typename Ret>
auto [lygintiElementusPagalDidejanciaReiksme](#) (Ret(T::*getter)() const)
Sukuria palyginimo funkciją rikiavimui didėjančia tvarka.
- template<typename T, typename Ret>
auto [lygintiElementusPagalMazejanciaReiksme](#) (Ret(T::*getter)() const)
Sukuria palyginimo funkciją rikiavimui mažėjančia tvarka.
- void [apskaiciuotiGalutiniPazymi](#) ([Studentas](#) &studentas, char skaiciavimoMetodoPasirinkimas)
Apskaiciuoja ir nustato vieno studento galutinį pažymį.
- void [apskaiciuotiGalutiniusPazymius](#) (std::vector< [Studentas](#) > &studentuSarasas, char skaiciavimoMetodoPasirinkimas)
Apskaiciuoja ir nustato galutinius pažymius visiems studentams.
- void [suskirstytiStudentus](#) (const std::vector< [Studentas](#) > &studentuSarasas, std::vector< [Studentas](#) > &pazangiuSarasas, std::vector< [Studentas](#) > &silpnuSarasas)
Suskirsto studentus į pažangius ir silpnus pagal galutinį pažymį.

5.17.1 Smulkus aprašymas

Funkcijos studentų pažymių skaičiavimui ir skirstymui.

5.17.2 Funkcijos Dokumentacija

5.17.2.1 apskaiciuotiGalutiniPazymi()

```
void apskaiciuotiGalutiniPazymi (
    Studentas & studentas,
    char skaiciavimoMetodoPasirinkimas)
```

Apskaičiuoja ir nustato vieno studento galutinį pažymį.

Parametrai

<i>studentas</i>	Studentas, kurio pažymys skaičiuojamas.
<i>skaiciavimoMetodoPasirinkimas</i>	Skaičiavimo metodo pasirinkimas.

5.17.2.2 apskaiciuotiGalutiniusPazymius()

```
void apskaiciuotiGalutiniusPazymius (
    std::vector< Studentas > & studentuSarasas,
    char skaiciavimoMetodoPasirinkimas)
```

Apskaičiuoja ir nustato galutinius pažymius visiems studentams.

Parametrai

<i>studentuSarasas</i>	Studentų sąrašas.
<i>skaiciavimoMetodoPasirinkimas</i>	Skaičiavimo metodo pasirinkimas.

5.17.2.3 lygintiElementusPagalDidejanciaReiksme()

```
template<typename T, typename Ret>
auto lygintiElementusPagalDidejanciaReiksme (
    Ret (T::* getter) () const)
```

Sukuria palyginimo funkciją rikiavimui didėjančia tvarka.

Template Parameters

<i>T</i>	Objekto tipas.
<i>Ret</i>	Getter funkcijos grąžinamas tipas.

Parametrai

<i>getter</i>	Klasės getter metodo rodyklė.
---------------	-------------------------------

Gražina

Lambda funkcija, tinkama rikiavimui didėjančia tvarka.

5.17.2.4 lygintiElementusPagalMazejanciaReiksme()

```
template<typename T, typename Ret>
auto lygintiElementusPagalMazejanciaReiksme (
    Ret(T::* getter)() const)
```

Sukuria palyginimo funkciją rikiavimui mažėjančia tvarka.

Template Parameters

<i>T</i>	Objekto tipas.
<i>Ret</i>	Getter funkcijos grąžinamas tipas.

Parametrai

<i>getter</i>	Klasės getter metodo rodyklė.
---------------	-------------------------------

Gražina

Lambda funkcija, tinkama rikiavimui mažėjančia tvarka.

5.17.2.5 skaiciuotiGalutineMediana()

```
double skaiciuotiGalutineMediana (
    const Studentas & studentas)
```

Apskaičiuoja galutinį pažymį pagal namų darbų medianą.

Parametrai

<i>studentas</i>	Studentas , kurio pažymys skaičiuojamas.
------------------	--

Gražina

Galutinis pažymys pagal medianą.

5.17.2.6 skaiciuotiGalutiniVidurki()

```
double skaiciuotiGalutiniVidurki (
    const Studentas & studentas)
```

Apskaičiuoja galutinį pažymį pagal namų darbų vidurkį.

Parametrai

<i>studentas</i>	Studentas , kurio pažymys skaičiuojamas.
------------------	--

Gražina

Galutinis pažymys pagal vidurkį.

5.17.2.7 skaiciuotiNDMediana()

```
double skaiciuotiNDMediana (
    std::vector< int > ndPazymiai)
```

Apskaičiuoja namų darbų pažymių medianą.

Parametrai

<i>ndPazymiai</i>	Namų darbų pažymių vektorius.
-------------------	-------------------------------

Gražina

Namų darbų pažymių mediana.

5.17.2.8 skaiciuotiNDVidurki()

```
double skaiciuotiNDVidurki (
    const std::vector< int > & ndPazymiai)
```

Apskaičiuoja namų darbų pažymių vidurkį.

Parametrai

<i>ndPazymiai</i>	Namų darbų pažymių vektorius.
-------------------	-------------------------------

Gražina

Namų darbų pažymių vidurkis.

5.17.2.9 suskirstytiStudentus()

```
void suskirstytiStudentus (
    const std::vector< Studentas > & studentuSarasas,
    std::vector< Studentas > & pazangiuSarasas,
    std::vector< Studentas > & silpnuSarasas)
```

Suskirsto studentus į pažangius ir silpnus pagal galutinį pažymį.

Studentai, kurių galutinis pažymys mažesnis nei 5.0, priskiriami silpniems. Likę studentai priskiriami pažangiems.

Parametrai

<i>studentuSarasas</i>	Pradinis studentų sąrašas.
<i>pazangiuSarasas</i>	Pažangių studentų sąrašas.
<i>silpnuSarasas</i>	Silpnų studentų sąrašas.

5.18 strukturaSkaiciavimai.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STRUKTURASKAICIAVIMAI_H
00002 #define STRUKTURASKAICIAVIMAI_H
00003
00004 #include <vector>
00005 #include "../Studentas.h"
00010
00016 double skaiciuotiNDVidurki(const std::vector<int>& ndPazymiai);
00017
00023 double skaiciuotiGalutiniVidurki(const Studentas& studentas);
00024
00030 double skaiciuotiNDMediana(std::vector<int> ndPazymiai);
00031
00037 double skaiciuotiGalutineMediana(const Studentas& studentas);
00046 template<typename T, typename Ret>
00047 auto lygintiElementusPagalDidejanciaReiksme(Ret (T::getter)() const) {
00048     return [getter](const T& a, const T& b) {
00049         return (a.*getter)() < (b.*getter)();
00050     };
00051 }
00060 template<typename T, typename Ret>
00061 auto lygintiElementusPagalMazejanciaReiksme(Ret (T::getter)() const) {
00062     return [getter](const T& a, const T& b) {
00063         return (a.*getter)() > (b.*getter)();
00064     };
00065 }
00071 void apskaiciuotiGalutiniPazymi(Studentas& studentas, char skaiciavimoMetodoPasirinkimas);
00077 void apskaiciuotiGalutiniusPazymius(std::vector<Studentas>& studentuSarasas, char
skaiciavimoMetodoPasirinkimas);
00088 void suskirstytiStudentus(const std::vector<Studentas>& studentuSarasas, std::vector<Studentas>&
pazangiuSarasas, std::vector<Studentas>& silpnuSarasas);
00089
00090 #endif
```

5.19 strukturosFailai/strukturaTestavimas.h Failo Nuoroda

Programos veikimo greičio, kontenerių strategijų ir [Studentas](#) klasės testavimo funkcijos.

```
#include "../strukturosFailai/Failai.h"
#include <chrono>
```

Klasės

- struct [TestoLaikai](#)
Saugo atskirų programos etapų vykdymo laikus.

Funkcijos

- double [apskaiciuotiBendraLaika](#) (const [TestoLaikai](#) &laikai)
Apskaiciuoja bendrą testavimo laiką.
- void [gautiVidurki](#) ([TestoLaikai](#) &laikai, int kiekis)
Apskaiciuoja vidutinius laikus pagal atliktų testų kieki.
- double [apskaiciuotiLaika](#) (std::chrono::steady_clock::time_point startas, std::chrono::steady_clock::time_point pabaiga)
Apskaiciuoja laiko skirtumą tarp dviejų momentų.
- template<typename Func>
double [ismatuotiLaika](#) (Func veiksmas)
Išmatuoja perduotos funkcijos arba veiksmo vykdymo laiką.

- void **vykdytiIvedimoTestavima** ()
Vykdo studentų įvedimo testavimą.
- void **vykdytiIšvedimoTestavima** (Failai & failai)
Vykdo studentų išvedimo testavimą.
- void **vykdytiDuomenuApdorojimoTestavima** ()
Vykdo bendrą duomenų apdorojimo testavimą.
- void **vykdytiNulintajaKonteineriuTestavimoStrategija** ()
Vykdo nulinę konteinerių testavimo strategiją.
- void **vykdytiPirmajaKonteineriuTestavimoStrategija** ()
Vykdo pirmąją konteinerių testavimo strategiją.
- void **vykdytiAntrajaKonteineriuTestavimoStrategija** ()
Vykdo antrąją konteinerių testavimo strategiją.
- void **vykdytiTreciajaKonteineriuTestavimoStrategija** ()
Vykdo trečiąją konteinerių testavimo strategiją.
- void **vykdytiTreciajaKonteineriuTestavimoStrategijaTikSuVektoriais** ()
Vykdo trečiąją konteinerių testavimo strategiją tik su `std::vector`.

Studentas klasės Google Test pagalbinės funkcijos

Atskiros funkcijos, kurias kviečia Google Test `TEST ( . . . )` blokai.

Šios funkcijos naudoja Google Test makrokomandas savo `.cpp` faile. Į šį `.h` failą `#include <gtest/↵ gtest.h>` dėti nereikia.

- void **testuotiDefaultKonstruktoriu** ()
Testuoja numatytąjį `Studentas` konstruktorių.
- void **testuotiParametriniKonstruktoriu** ()
Testuoja parametrinį `Studentas` konstruktorių.
- void **testuotiKonstruktoriusEilutes** ()
Testuoja `Studentas` konstruktorių, kuris sukuria objektą iš tekstinės eilutės.
- void **testuotiSetteriusIrGetterius** ()
Testuoja `Studentas` setterius ir getterius.
- void **testuotiNamuDarbuPazymiuValdyma** ()
Testuoja namų darbų pažymių pridėjimą, rezervavimą ir išvalymą.
- void **testuotiGalutinioPazymioSkaiciavimaPagalVidurki** ()
Testuoja galutinio pažymio skaičiavimą pagal namų darbų vidurkį.
- void **testuotiCopyKonstruktoriu** ()
Testuoja `Studentas` kopijavimo konstruktorių.
- void **testuotiCopyPriskyrimoOperatoriu** ()
Testuoja `Studentas` kopijavimo priskyrimo operatorių.
- void **testuotiSelfCopyAssignment** ()
Testuoja savęs priskyrimo atvejį naudojant kopijavimo priskyrimo operatorių.
- void **testuotiMoveKonstruktoriu** ()
Testuoja `Studentas` perkėlimo konstruktorių.
- void **testuotiMovePriskyrimoOperatoriu** ()
Testuoja `Studentas` perkėlimo priskyrimo operatorių.
- void **testuotiIšvestiesOperatoriu** ()
Testuoja `Studentas` išvesties operatorių `operator<<`.
- void **testuotiĮvestiesOperatoriu** ()
Testuoja `Studentas` įvesties operatorių `operator>>`.
- void **testuotiStudentasLenteIšvestiesOperatoriu** ()
Testuoja `StudentasLenteI` išvesties operatorių `operator<<`.
- void **testuotiPaveldimumoStruktura** ()
Testuoja paveldėjimo ryšį tarp `Zmogus` ir `Studentas`.
- void **testuotiDestruktoriu** ()
Testuoja, ar `Studentas` destruktoriaus išskviečiamas ir sukuria tikėtiną šalutinį efektą.

5.19.1 Smulkus aprašymas

Programos veikimo greičio, konteinerių strategijų ir [Studentas](#) klasės testavimo funkcijos.

5.19.2 Funkcijos Dokumentacija

5.19.2.1 apskaiciuotiBendraLaika()

```
double apskaiciuotiBendraLaika (
    const TestoLaikai & laikai)
```

Apskaičiuoja bendrą testavimo laiką.

Parametrai

<i>laikai</i>	Struktūra su atskirų etapų laikais.
---------------	-------------------------------------

Gražina

Bendras laikas sekundėmis.

5.19.2.2 apskaiciuotiLaika()

```
double apskaiciuotiLaika (
    std::chrono::steady_clock::time_point startas,
    std::chrono::steady_clock::time_point pabaiga)
```

Apskaičiuoja laiko skirtumą tarp dviejų momentų.

Parametrai

<i>startas</i>	Pradžios laiko momentas.
<i>pabaiga</i>	Pabaigos laiko momentas.

Gražina

Laiko skirtumas sekundėmis.

5.19.2.3 gautiVidurki()

```
void gautiVidurki (
    TestoLaikai & laikai,
    int kiekis)
```

Apskaičiuoja vidutinius laikus pagal atliktų testų kiekį.

Parametrai

<i>laikai</i>	Laikų struktūra, kurios reikšmės bus padalintos iš testų kiekio.
---------------	--

<i>kiekis</i>	Testų kiekis.
---------------	---------------

5.19.2.4 ismatuotiLaika()

```
template<typename Func>
double ismatuotiLaika (
    Func veiksmas)
```

Išmatuoja perduotos funkcijos arba veiksmo vykdymo laiką.

Template Parameters

<i>Func</i>	Funkcijos arba lambda tipas.
-------------	------------------------------

Parametrai

<i>veiksmas</i>	Veiksmas, kurio vykdymo laikas matuojamas.
-----------------	--

Gražina

Vykdyto laiko sekundėmis.

5.19.2.5 vykdytiIšvedimoTestavima()

```
void vykdytiIšvedimoTestavima (
    Failai & failai)
```

Vykdo studentų išvedimo testavimą.

Parametrai

<i>failai</i>	Vardų ir pavardžių failų duomenys.
---------------	------------------------------------

5.20 strukturaTestavimas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STRUKTURATESTAVIMAS_H
00002 #define STRUKTURATESTAVIMAS_H
00003
00004 #include "../strukturosFailai/Failai.h"
00005 #include <chrono>
00006
00011
00016 struct TestoLaikai {
00018     double nuskaitymas = 0.0;
00019
00021     double skaiciavimas = 0.0;
00022
00024     double rikiavimas = 0.0;
00025
00027     double skirstymas = 0.0;
```

```

00028
00030     double isvedimas = 0.0;
00031 };
00032
00038 double apskaiciuotiBendraLaika(const TestoLaikai& laikai);
00039
00045 void gautiVidurki(TestoLaikai& laikai, int kiekis);
00046
00053 double apskaiciuotiLaika(
00054     std::chrono::steady_clock::time_point startas,
00055     std::chrono::steady_clock::time_point pabaiga
00056 );
00057
00064 template <typename Func>
00065 double ismatuotiLaika(Func veiksmas) {
00066     auto pradzia = std::chrono::steady_clock::now();
00067     veiksmas();
00068     auto pabaiga = std::chrono::steady_clock::now();
00069     return apskaiciuotiLaika(pradzia, pabaiga);
00070 }
00071
00075 void vykdytiIvedimoTestavima();
00076
00081 void vykdytiIsvedimoTestavima(Failai& failai);
00082
00086 void vykdytiDuomenuApdorojimoTestavima();
00087
00091 void vykdytiNulintajaKonteineriuTestavimoStrategija();
00092
00096 void vykdytiPirmajaKonteineriuTestavimoStrategija();
00097
00101 void vykdytiAntrajaKonteineriuTestavimoStrategija();
00102
00106 void vykdytiTreciajaKonteineriuTestavimoStrategija();
00107
00111 void vykdytiTreciajaKonteineriuTestavimoStrategijaTikSuVektoriais();
00112
00121
00125 void testuotiDefaultKonstruktoriu();
00126
00130 void testuotiParametriniKonstruktoriu();
00131
00135 void testuotiKonstruktoriuIsEilutes();
00136
00140 void testuotiSetteriusIrGetterius();
00141
00145 void testuotiNamuDarbuPazymiuValdyma();
00146
00150 void testuotiGalutinioPazymioSkaiciavimaPagalVidurki();
00151
00155 void testuotiCopyKonstruktoriu();
00156
00160 void testuotiCopyPriskyrimoOperatoriu();
00161
00165 void testuotiSelfCopyAssignment();
00166
00170 void testuotiMoveKonstruktoriu();
00171
00175 void testuotiMovePriskyrimoOperatoriu();
00176
00180 void testuotiIsvestiesOperatoriu();
00181
00185 void testuotiIvestiesOperatoriu();
00186
00190 void testuotiStudentasLenteleiIsvestiesOperatoriu();
00191
00195 void testuotiPaveldimumoStruktura();
00196
00200 void testuotiDestruktoriu();
00201
00202 #endif

```

5.21 struktūrosFailai/strukturaVisi.h Failo Nuoroda

Bendras antraštinis failas, įtraukiantis visus pagrindinius projekto modulius.

```

#include "../struktūrosFailai/strukturaDarbasSuFailais.h"
#include "../Studentas.h"
#include "../struktūrosFailai/strukturaGeneravimas.h"

```

```
#include "../strukturosFailai/strukturaIvestis.h"
#include "../strukturosFailai/strukturaIsvestis.h"
#include "../strukturosFailai/strukturaSkaiciavimai.h"
#include "../strukturosFailai/strukturaRikiavimas.h"
#include "../strukturosFailai/Failai.h"
#include "../strukturosFailai/strukturaMenu.h"
#include "../strukturosFailai/strukturaTestavimas.h"
#include "../strukturosFailai/SabloninesFunkcijos.h"
```

5.21.1 Smulkus aprašymas

Bendras antraštinis failas, įtraukiantis visus pagrindinius projekto modulius.

Šis failas naudojamas patogiam visų svarbiausių projekto struktūrų ir funkcijų įtraukimui vienoje vietoje.

5.22 strukturaVisi.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STRUKTURAVISI_H
00002 #define STRUKTURAVISI_H
00003
00011
00012 #include "../strukturosFailai/strukturaDarbasSuFailais.h"
00013 #include "../Studentas.h"
00014 #include "../strukturosFailai/strukturaGeneravimas.h"
00015 #include "../strukturosFailai/strukturaIvestis.h"
00016 #include "../strukturosFailai/strukturaIsvestis.h"
00017 #include "../strukturosFailai/strukturaSkaiciavimai.h"
00018 #include "../strukturosFailai/strukturaRikiavimas.h"
00019 #include "../strukturosFailai/Failai.h"
00020 #include "../strukturosFailai/strukturaMenu.h"
00021 #include "../strukturosFailai/strukturaTestavimas.h"
00022 #include "../strukturosFailai/SabloninesFunkcijos.h"
00023
00024
00025 #endif
```

5.23 Studentas.h Failo Nuoroda

Studentą aprašanti klasė, paveldinti bazinę [Zmogus](#) klasę.

```
#include <iostream>
#include <string>
#include <vector>
#include "Zmogus.h"
```

Klasės

- class [Studentas](#)
Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.
- struct [StudentasLentelei](#)
Pagalbinė struktūra studento išvedimui lentelės formatu.

Funkcijos

- `std::ostream & operator<< (std::ostream &os, const StudentasLentelei &s)`
Išveda studentą lentelės formatu.

5.23.1 Smulkus aprašymas

Studentą aprašanti klasė, paveldinti bazinę `Zmogus` klasę.

Šiame faile aprašoma `Studentas` klasė, kuri saugo studento vardą, pavardę, egzamino pažymį, namų darbų pažymius ir galutinį pažymį. Taip pat aprašomi įvesties, išvesties, kopijavimo ir perkėlimo operatoriai.

5.23.2 Funkcijos Dokumentacija

5.23.2.1 `operator<<()`

```
std::ostream & operator<< (
    std::ostream & os,
    const StudentasLentelei & s)
```

Išveda studentą lentelės formatu.

Parametrai

<code>os</code>	Išvesties srautas.
<code>s</code>	Pagalbinė struktūra su studento nuoroda.

Gražina

Išvesties srautas.

5.24 Studentas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include "Zmogus.h"
00008
00017
00026 class Studentas : public Zmogus {
00027 private:
00029     int examGrade_;
00030
00032     std::vector<int> homeworkGrades_;
00033
00035     double finalGrade_;
00036 public:
00037     public:
00041     static bool arSpausdintiDestruktoriu;
00042
00048     Studentas() : Zmogus(), examGrade_(0), finalGrade_(0) {}
00049
```

```

00058     Studentas(const std::string& name,
00059               const std::string& surname,
00060               int examGrade,
00061               const std::vector<int>& homeworkGrades,
00062               double finalGrade)
00063     : Zmogus(name, surname),
00064       examGrade_(examGrade),
00065       homeworkGrades_(homeworkGrades),
00066       finalGrade_(finalGrade) {}
00067
00076     Studentas(const char* eilute, std::size_t namuDarbuKiekis);
00077
00083     Studentas(bool generuotiPazymius, int ndKiekis);
00090     ~Studentas();
00091
00096     Studentas(const Studentas& other);
00097
00103     Studentas& operator=(const Studentas& other);
00104
00109     Studentas(Studentas&& other) noexcept;
00110
00116     Studentas& operator=(Studentas&& other) noexcept;
00117
00122     inline std::string getName() const { return name_; }
00123
00128     inline std::string getSurname() const { return surname_; }
00133     inline double getFinalGrade() const { return finalGrade_; }
00134
00139     inline int getExamGrade() const { return examGrade_; }
00140
00145     inline const std::vector<int>& getHomeworkGrades() const { return homeworkGrades_; }
00146
00156     double calculateFinalGrade(char method) const;
00157
00162     void setName(const std::string& name) { name_ = name; }
00163
00168     void setSurname(const std::string& surname) { surname_ = surname; }
00169
00174     void setExamGrade(int examGrade) { examGrade_ = examGrade; }
00179     void setFinalGrade(double finalGrade) { finalGrade_ = finalGrade; }
00180
00185     void setHomeworkGrades(const std::vector<int>& homeworkGrades) { homeworkGrades_ = homeworkGrades; }
00186
00191     void addHomeworkGrade(int grade) { homeworkGrades_.push_back(grade); }
00192
00196     void clearHomeworkGrades() { homeworkGrades_.clear(); }
00197
00202     void reserveHomeworkGrades(std::size_t n) { homeworkGrades_.reserve(n); }
00203
00210     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00211
00218     friend std::istream& operator>>(std::istream& is, Studentas& s);
00219 };
00220
00228 struct StudentasLentelei {
00230     const Studentas& s;
00231 };
00232
00239 std::ostream& operator<<(std::ostream& os, const StudentasLentelei& s);
00240
00241 #endif

```

5.25 Zmogus.h Failo Nuoroda

Abstrakti bazinė žmogaus klasė.

```
#include <string>
```

Klasės

- class [Zmogus](#)

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

5.25.1 Smulkus aprašymas

Abstrakti bazinė žmogaus klasė.

Šiame faile aprašoma abstrakti bazinė klasė [Zmogus](#). Ji saugo bendrus žmogaus duomenis: vardą ir pavardę. Klasė skirta paveldėjimui, todėl tiesiogiai [Zmogus](#) objektai nekuriami.

5.26 Zmogus.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00014
00022 class Zmogus {
00023 protected:
00027     std::string name_;
00028
00032     std::string surname_;
00033
00034 public:
00040     Zmogus() = default;
00041
00048     Zmogus(const std::string& name, const std::string& surname) : name_(name), surname_(surname) {}
00049
00059     virtual ~Zmogus() = 0;
00060
00066     virtual std::string getName() const = 0;
00067
00073     virtual std::string getSurname() const = 0;
00074 };
00075
00082 inline Zmogus::~Zmogus() {}
00083
00084 #endif
```


Rodyklė

~Studentas
Studentas, [11](#)
~Zmogus
Zmogus, [20](#)

addHomeworkGrade
Studentas, [12](#)
apdorotiIrsvestiStudentus
strukturalSvestis.h, [36](#)
apskaiciuotiBendraLaika
strukturaTestavimas.h, [59](#)
apskaiciuotiGalutiniPazymi
strukturaSkaiciavimai.h, [54](#)
apskaiciuotiGalutiniusPazymius
SabloninesFunkcijos.h, [23](#)
strukturaSkaiciavimai.h, [54](#)
apskaiciuotiLaika
strukturaTestavimas.h, [59](#)

calculateFinalGrade
Studentas, [12](#)

Failai, [7](#)

gautiNuskaitymoMeniu
strukturaMeniu.h, [48](#)
gautiTekstiniusFailus
strukturaDarbasSuFailais.h, [29](#)
gautiVidurki
strukturaTestavimas.h, [59](#)
generuotiRezultatus
strukturaGeneravimas.h, [34](#)
generuotiStudentus
strukturaGeneravimas.h, [34](#)
generuotiSveikaSkaiciu
strukturaGeneravimas.h, [34](#)
generuotiVardaPavarde
strukturaGeneravimas.h, [35](#)
getExamGrade
Studentas, [12](#)
getFinalGrade
Studentas, [13](#)
getHomeworkGrades
Studentas, [13](#)
getName
Studentas, [13](#)
Zmogus, [20](#)
getSurname
Studentas, [13](#)
Zmogus, [20](#)

irasytiDuomenis
strukturaDarbasSuFailais.h, [29](#)
irasytiStudentuDuomenisIFaila
strukturaDarbasSuFailais.h, [30](#)
strukturalSvestis.h, [37](#)
irasytiSuskirstytusStudentusIFailus
strukturaDarbasSuFailais.h, [30](#)
ismatuotiLaika
strukturaTestavimas.h, [60](#)
isvestiStudentus
strukturalSvestis.h, [37](#)

lygintiElementusPagalDidejanciaReiksme
strukturaSkaiciavimai.h, [54](#)
lygintiElementusPagalMazejanciaReiksme
strukturaSkaiciavimai.h, [54](#)

nuskaitytiDuomenis
strukturaDarbasSuFailais.h, [30](#)
nuskaitytiDuomenisGeneric
SabloninesFunkcijos.h, [23](#)
nuskaitytiEilutesIVektoriu
strukturaDarbasSuFailais.h, [31](#)
nuskaitytiMeniuPasirinkima
strukturalvestis.h, [40](#)
strukturaMeniu.h, [48](#), [49](#)
nuskaitytiNamuDarbuPazymius
strukturalvestis.h, [41](#)
nuskaitytiNeneigiamaSveikajiSkaiciu
strukturalvestis.h, [41](#)
nuskaitytiPagrindinioMeniuPasirinkima
strukturalvestis.h, [41](#)
strukturaMeniu.h, [49](#)
nuskaitytiPazymiNuo1iki10
strukturalvestis.h, [42](#)
nuskaitytiSkaiciavimoMetoda
strukturalvestis.h, [42](#)
nuskaitytiStudentuDuomenisIsFailo
SabloninesFunkcijos.h, [23](#)
strukturaDarbasSuFailais.h, [31](#)
nuskaitytiSveikajiSkaiciu
strukturalvestis.h, [42](#)
nuskaitytiSveikaSkaiciusFailo
strukturaDarbasSuFailais.h, [31](#)
nuskaitytiTeigiamaSveikajiSkaiciu
strukturalvestis.h, [42](#)
nuskaitytiVardaArPavarde
strukturalvestis.h, [43](#)
nuskaitytiZodisIsFailo
strukturaDarbasSuFailais.h, [32](#)

operator<<
 Studentas, 16
 Studentas.h, 63
 operator>>
 Studentas, 16
 operator=
 Studentas, 14

 parinktiRikiavimoBudus
 strukturalvestis.h, 43
 parodytiRezultatuLentele
 strukturalvestis.h, 37
 parodytiStudentus
 strukturalvestis.h, 38
 patvirtintiNaujoStudentoPridejima
 strukturalvestis.h, 43
 perkeltiStudentus
 SabloninesFunkcijos.h, 24
 praleistiTarpalsFailo
 strukturaDarbasSuFailais.h, 32

 reserveHomeworkGrades
 Studentas, 14
 rikiuotiStudentus
 SabloninesFunkcijos.h, 24
 strukturaRikiavimas.h, 52
 rikiuotiSuskirstytusStudentus
 SabloninesFunkcijos.h, 25
 strukturaRikiavimas.h, 52

 SabloninesFunkcijos.h
 apskaiciuotiGalutiniusPazymius, 23
 nuskaitytiDuomenisGeneric, 23
 nuskaitytiStudentuDuomenisIsFailo, 23
 perkeltiStudentus, 24
 rikiuotiStudentus, 24
 rikiuotiSuskirstytusStudentus, 25
 skirstytiIstrinantStudentus, 26
 skirstytiIstrinantStudentusEfektyviau, 26
 saugiaiNuskaitytiEilute
 strukturalvestis.h, 44
 setExamGrade
 Studentas, 14
 setFinalGrade
 Studentas, 15
 setHomeworkGrades
 Studentas, 15
 setName
 Studentas, 15
 setSurname
 Studentas, 15
 skaiciuotiGalutineMediana
 strukturaSkaiciavimai.h, 55
 skaiciuotiGalutiniVidurki
 strukturaSkaiciavimai.h, 55
 skaiciuotiNDMediana
 strukturaSkaiciavimai.h, 55
 skaiciuotiNDVidurki
 strukturaSkaiciavimai.h, 55

 skirstytiIstrinantStudentus
 SabloninesFunkcijos.h, 26
 skirstytiIstrinantStudentusEfektyviau
 SabloninesFunkcijos.h, 26
 spausdintiVidurkius
 strukturalvestis.h, 38
 strukturaDarbasSuFailais.h
 gautiTekstiniusFailus, 29
 irasytiDuomenis, 29
 irasytiStudentuDuomenisIsFaila, 30
 irasytiSuskirstytusStudentusIsFailus, 30
 nuskaitytiDuomenis, 30
 nuskaitytiEilutesIvektoriu, 31
 nuskaitytiStudentuDuomenisIsFailo, 31
 nuskaitytiSveikaSkaiciulsFailo, 31
 nuskaitytiZodilsFailo, 32
 praleistiTarpalsFailo, 32
 vykdytiSkirstymaIsFailus, 32
 strukturaGeneravimas.h
 generuotiRezultatus, 34
 generuotiStudentus, 34
 generuotiSveikaSkaiciu, 34
 generuotiVardaPavarde, 35
 strukturalvestis.h
 apdorotiIIsvestiStudentus, 36
 irasytiStudentuDuomenisIsFaila, 37
 isvestiStudentus, 37
 parodytiRezultatuLentele, 37
 parodytiStudentus, 38
 spausdintiVidurkius, 38
 vykdytiIrasymaIsFaila, 38
 strukturalvestis.h
 nuskaitytiMenuPasirinkima, 40
 nuskaitytiNamuDarbuPazymius, 41
 nuskaitytiNeneigiamasveikajiSkaiciu, 41
 nuskaitytiPagrindinioMenuPasirinkima, 41
 nuskaitytiPazymiNuo1iki10, 42
 nuskaitytiSkaiciavimoMetoda, 42
 nuskaitytiSveikajiSkaiciu, 42
 nuskaitytiTeigiamasveikajiSkaiciu, 42
 nuskaitytiVardaArPavarde, 43
 parinktiRikiavimoBudus, 43
 patvirtintiNaujoStudentoPridejima, 43
 saugiaiNuskaitytiEilute, 44
 suformuotiStudentolvestiesEilute, 44
 sukurtiStudentalsEilutesPerOperatoriu, 44
 tikrintilvesti, 45
 tvarkytiPavarde, 45
 tvarkytiVarda, 45
 vykdytiPilnaGeneravima, 45
 vykdytiStudentulvedima, 46
 strukturaMenu.h
 gautiNuskaitymoMenu, 48
 nuskaitytiMenuPasirinkima, 48, 49
 nuskaitytiPagrindinioMenuPasirinkima, 49
 strukturaRikiavimas.h
 rikiuotiStudentus, 52
 rikiuotiSuskirstytusStudentus, 52

- strukturaSkaiciavimai.h
 - apskaiciuotiGalutiniPazymi, [54](#)
 - apskaiciuotiGalutiniusPazymius, [54](#)
 - lygintiElementusPagalDidejanciaReiksme, [54](#)
 - lygintiElementusPagalMazejanciaReiksme, [54](#)
 - skaiciuotiGalutineMediana, [55](#)
 - skaiciuotiGalutiniVidurki, [55](#)
 - skaiciuotiNDMediana, [55](#)
 - skaiciuotiNDVidurki, [56](#)
 - suskirstytiStudentus, [56](#)
- strukturaTestavimas.h
 - apskaiciuotiBendraLaika, [59](#)
 - apskaiciuotiLaika, [59](#)
 - gautiVidurki, [59](#)
 - ismatuotiLaika, [60](#)
 - vykdytilsvedimoTestavima, [60](#)
- strukturosFailai/Failai.h, [21](#)
- strukturosFailai/SabloninesFunkcijos.h, [22](#), [27](#)
- strukturosFailai/strukturaDarbasSuFailais.h, [28](#), [33](#)
- strukturosFailai/strukturaGeneravimas.h, [33](#), [35](#)
- strukturosFailai/strukturalvestis.h, [36](#), [39](#)
- strukturosFailai/strukturalvestis.h, [39](#), [46](#)
- strukturosFailai/strukturaMenu.h, [47](#), [50](#)
- strukturosFailai/strukturaRikiavimas.h, [51](#), [53](#)
- strukturosFailai/strukturaSkaiciavimai.h, [53](#), [57](#)
- strukturosFailai/strukturaTestavimas.h, [57](#), [60](#)
- strukturosFailai/strukturaVisi.h, [61](#), [62](#)
- Studentas, [7](#)
 - ~Studentas, [11](#)
 - addHomeworkGrade, [12](#)
 - calculateFinalGrade, [12](#)
 - getExamGrade, [12](#)
 - getFinalGrade, [13](#)
 - getHomeworkGrades, [13](#)
 - getName, [13](#)
 - getSurname, [13](#)
 - operator<<, [16](#)
 - operator>>, [16](#)
 - operator=, [14](#)
 - reserveHomeworkGrades, [14](#)
 - setExamGrade, [14](#)
 - setFinalGrade, [15](#)
 - setHomeworkGrades, [15](#)
 - setName, [15](#)
 - setSurname, [15](#)
 - Studentas, [9](#), [11](#), [12](#)
- Studentas.h, [62](#)
 - operator<<, [63](#)
- StudentasLentelei, [17](#)
- suformuotiStudentolvestiesEilute
 - strukturalvestis.h, [44](#)
- sukurtiStudentalsEilutesPerOperatoriu
 - strukturalvestis.h, [44](#)
- suskirstytiStudentus
 - strukturaSkaiciavimai.h, [56](#)
- TestoLaikai, [17](#)
- tikrintilvesti
 - strukturalvestis.h, [45](#)
- tvarkytiPavarde
 - strukturalvestis.h, [45](#)
- tvarkytiVarda
 - strukturalvestis.h, [45](#)
- vykdytilrasymaFaila
 - strukturalvestis.h, [38](#)
- vykdytilsvedimoTestavima
 - strukturaTestavimas.h, [60](#)
- vykdytiPilnaGeneravima
 - strukturalvestis.h, [45](#)
- vykdytiSkirstymaFailus
 - strukturaDarbasSuFailais.h, [32](#)
- vykdytiStudentulvedima
 - strukturalvestis.h, [46](#)
- Zmogus, [18](#)
 - ~Zmogus, [20](#)
 - getName, [20](#)
 - getSurname, [20](#)
 - Zmogus, [18](#)
- Zmogus.h, [64](#)